University of Bergen
Department of Informatics

Performance of Distributed and Shared Memory Parallel Sparse Matrix Vector Multiplication

Author: Kristian Sørdal
Supervisor: Johannes Langguth

UNIVERSITETET I BERGEN
Fakultet for naturvitenskap og teknologi

May, 2025

To err is human; but to really foul things up requires a computer.

Paul R. Ehrlich

Abstract

Sparse Matrix Vector Multiplication is an important kernel used in scientific computing. It is a problem that lends itself well to parlallization. The problem is bounded by, and scales with the memory bandwidth of the system. Therefore in order to efficiently perform *SpMV* on large distributed memory systems, it is important to reduce the communication between nodes, in order to extract as much as possible out of the memory bandwidth of the system.

This thesis aims to investigate the results of *SpMV* when ran using different communication strategies.

Acknowledgements

I would like to express my gratitude to my supervisor Johannes Langguth, for the helpful guidance, valuable discussions and insighful feedback he provided throughout the process of writing this thesis.

I would also like to thank my fellow students and friends in the Algorithms study hall, for all the fun times, frustrations shared, discussions had, and general shenanigans every day.

Finally, to my family and friends, with whom I've spent much of my spare time with.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Sparse-Matrix Vector Multiplication (SpMV) is a fundamental computational kernel in many scientific computing applications. It appears in a broad range of domains, such as iterative solvers for partial differntial equations, machine learning, and when simulating physical systems. As a result, optimizing the performance of SpMV, both on CPUs and GPUs remains a widely researched topic. SpMV lends itself nicely to paralellization, but it is a kernel with an inherently low computational intensity, and its performance is therefore memory bound.

This thesis aims to investigate the impact that different communication strategies have on the performance of SpMV. The experiments conducted in this thesis have been conducted using a distributed and shared memory hybrid parallel environment. To facilitate inter node communication, one MPI rank has been placed on each compute node (or on each socket if the compute node is dual-socketed), and shared memory paralleization with OpenMP has been used for intra node communication.

Four distinct communication strategies have been tested, with an additional fifth strategy that is fully memory scalable. These communication strategies successively become more sophisticated. The simplest being a strategy which communicates the entire local part of the dense x vector to each other rank. Then two separator based strategies, which involves communicating only the set of boundary elements that induce data-dependencies between nodes. Finally, the most sophisticated communication strategy involves communicating only the elements that induce data-dependencies to the node where this data-dependency occurs.

Each communication strategy is tested on a set of modern multicore architectures available on Simula's *Exploration of Experimental Exascale computing* (eX^3), using a set of well structured sparse matrices.

Chapter 2

Theory

2.1 Sparse Matrices

There are many different opinions on what is considered a sparse matrix, and there exists no definition that captures what a sparse matrix is. For this thesis, a matrix is considered sparse if it is worthwhile to treat nonzeros differently than zeros. Usually this means that for an $n \times n$ matrix, each row has a constant ($\mathcal{O}(1)$) number of nonzero entries per row. Or in other words, a $n \times n$ matrix is sparse if it has $\mathcal{O}(n)$ nonzero entires.

Given a sparse matrix A, an input vector x and the resulting output vector y, Matrix Vector Multiplication is defined as

$$y = Ax$$

where the y_i is given by $y_i = \sum_{j=1}^{n} a_{ij} x_j$.

2.2 Sparse Matrix Vector Multiplication

Sparse-Matrix Vector Multiplication (SpMV) can benefit from paralellization, both using shared memory and distributed memory, or a hybrid of both. Despite this, it is notoriosly difficult to optimize, sequentially and in a parallel setting. This is especially true for the large matrices that appear when solving problems in scientific computing.

As mentioned in the introduction, SpMV has an inherently low computational intensity, and its performance is therfore memory bound. This means that the memory bandwidth of the system becomes the primary bottleneck when scaling to multiple compute nodes. It becomes important

to carefully manage data dependencies between processes, as performance is directly correlated with the communication volume across the interconnects between nodes. This holds true regardless of the hardware that is used, and most of the challenges discussed in this thesis need to be taken into consideration when implementing SpMV on CPUs, GPUs, or other types of architecture.

2.3 Latency Numbers

The execution time of computational operations can vary significantly, and it is important to have some understanding of the latency times associated with typical operations. Referencing these latency numbers can be valuable when interpreting benchmark results and the performance of different programs.

Operation	**Time [ns]**
L1 cache reference	1
Branch misprediction	3
L2 cache reference	4
Mutex lock/unlock	17
Main memory reference	100
Compress 1K bytes with Zippy	2000
Send 2kB over 10 Gbps network	1600
Send 1K bytes over 1 Gbps network	10 000
Read 4K bytes randomly from SSD*	20 000
Round trip within same datacenter	500 000
Read 1MB sequentially from memory	1 000 000
Disk seek	10 000 000
Read 1MB sequentially form disk	10 000 000
TCP packet round trip between continents	150 000 000

Table 2.1: Latency numbers for common operation, adapted from [11].

2.4 Parallel Architectures

There are two main architectures used in the parallel computing industry: Shared Memory Architecture and Distributed Memory Architecture. The following sections gives an overview of the key difference between the two.

2.4.1 Shared Memory Architecture

On a system with shared memory architecture, every processing unit (PU) have access to the same memory, and treat it as a global address space. Although each thread maintains its own private cache for performance, reads and writes target the same shared memory pool. To guarantee correctness, the system must enforce *cache coherence*: any read must observe the most recent write, regardless of which cache holds that value. This is managed through what is called a coherence protocol, without which, threads could see stale or inconsistent data, leading to race conditions and incorrect program behavior [12].

2.4.2 Distributed Memory Architecture

On systems with distributed memory architecture, every processor have their own local memory, not accesible by other processors. When a process needs to access memory from another process, explicit communication of the data stored at that memory address needs to occur, and happens through whichever network the processors are connected with (adapted from [9]). Figure 2.1 illustrates the difference between shared and distributed memory architectures.

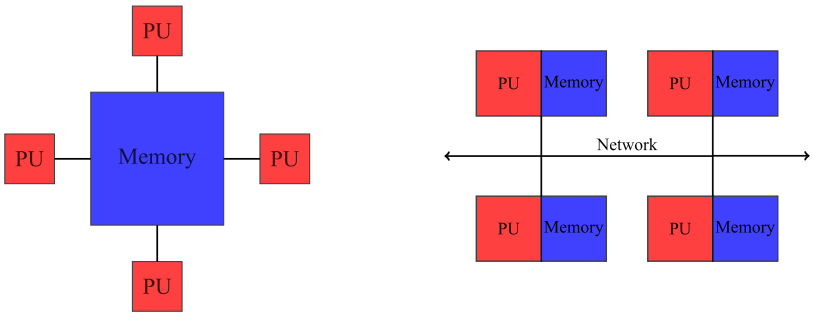


Figure 2.1: Shared and Distributed Memory Architecture (adapted from [8])

2.4.3 Non-Uniform Memory Access

On multiprocessor systems, Non-Uniform Memory Access (NUMA) memory architecture that lays out memory in such a way that the memory access latency depends on the physical distance between the processor core, and the location of the memory being accessed. On these systems each processor has its own local memory, which cores on that processor can access much faster than memory local to another processor. Figure 2.2 illustrates a system with two NUMA nodes, and if a core on node 0 needs to access memory on node 1, it needs to travel through the interconnect that connects the two nodes, which incurs higher latency.

Single chips can also partition their cores into multiple NUMA domains, as an example, a 32-core chip may have 4 NUMA domains where every domain contains eight nodes each. Respecting the NUMA domains is particularly important if performance is the goal, and can be achieved using tools such as `numactl`.

It is important to note that NUMA refers specifically to the organization of the main memory, i.e. the DRAM, and not to the on chip caches implemented using SRAM.

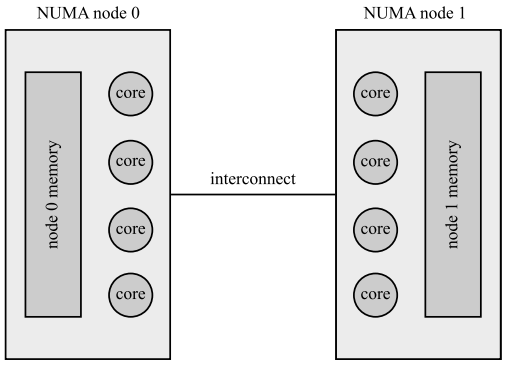


Figure 2.2: System with two NUMA nodes and eight processors (4 per NUMA node) (adapted from [7]).

2.4.4 First-Touch Policy

An important implication of the NUMA memory architecture is how and where memory pages are allocated. This is goverend by what is known as the First-Touch Policy, which is the default memory allocation policy Linux and other operating systems. According to this policy, a memory page is allocated in the memory that is closest to the thread that first requires this memory page. As an example, using Figure 2.2 as reference, if a core on NUMA node 0 is the first to access some memory page, it will be allocated in node 0's memory.

2.4.5 Emerging Architectures

In the ever evolving world of computer chip manufacturing, there has in the past years emerged a new trend in the field of architecture design. This involves packing thousands of small processor cores into a single device, where each core has its own local memory, and no device level shared memory. An example of such a processor is the relatively new Cerebras Wafer-Scale Engine (WSE-3), which is the largest chip ever built. With a spec sheet of 4 trillion transistors, 900 000 cores, memory bandwidth of 21PB/s, and 44 GB of on-wafer memory (see [5]). Such architectures necessitate a careful distribution of relevant data across each processor, and is where memory scalable communication strategies are beneficial. Another example is Graphcores Colossus MK2 GC200 IPU (Intelligence Processing Unit). Although not as big as WSE-3, it still contains 59.4 billion transistors and has 1472 cores.

Chapter 3

Background

3.1 CSR Storage Format

CSR (Compressed Sparse Row) is the most widely used storage format for sparse matrices. As its name suggests, it compresses the amount of memory used to store a matrix without loss of information. It does so by utilizing three vectors A_p, A_j, A_x. Figure 3.1 shows an example of a matrix stored in CSR format, adapted from [4].

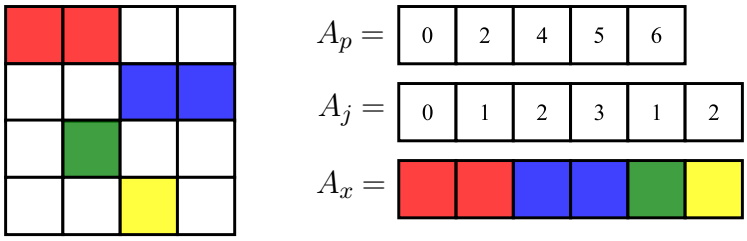


Figure 3.1: Matrix represented in the CSR format.

The first vector, A_p stores the indices of the first non-zero in the vectors A_p and A_x. For a given entry $A_p[i]$, $A_p[i]$ is the index of the first non-zero in the i^{th} row. $A_j[j]$ and $A_x[j]$ denotes the column index and value of the j^{th} non-zero, respectively.

Throughout the remainder of this thesis, we operate under the assumption that all matrices are represented in CSR format, unless explicitly noted otherwise.

3.1.1 Computational Intensity

The *computational intensity* of an operation is a key concept in performance analysis, and is defined as the ratio of the number of FLOPS to the number of memory accesses.

$$\text{Computational intensity} = \frac{\text{FLOPS}}{\text{Memory accesses}} \tag{3.1}$$

This metric helps determine whether a computation is *compute bound* or *memory bound*. Operations with high computational intensity implies that it can benefit from employing faster processors, as it performs a higher number of calculation per byte of memory read. Operations with lower computational intensity implies that the operation in memory bound, and thus employing faster processors wont necessarily yield higher performance, as the limiting factor is not the speed at which the calculations are performed, but rather the speed at which the required memory to perform the operation can be accessed at.

3.2 Other storage formats

While the CSR format is the most widely used storage format for sparse matrices, there exists sever other storage formats, each with its own benefits and drawbacks. In this section, some of these formats are briefly presented.

3.2.1 COO Format

The Coordinate List (COO) format stores the matrix as a set of triples on the form i, j, x, where i is the row index, j is the column index, and x is the value stored at (i, j) in the matrix. In this format all 0 entries are ignored.

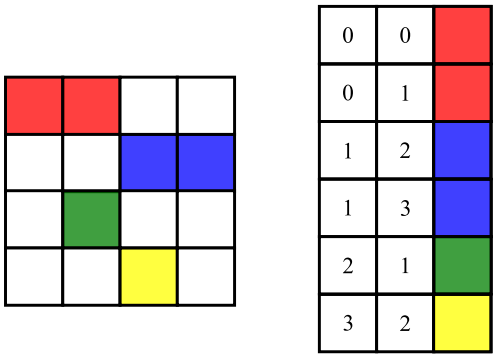


Figure 3.2: Example Matrix represented in the COO format.

3.2.2 CSC Format

The Compressed Sparse Column (CSC) format is similar to the CSR format, but instead of compressing the rows, we compress the columns. In this matrix format, we have irregular memory writes, but the reads are more regular. This can however be a problem

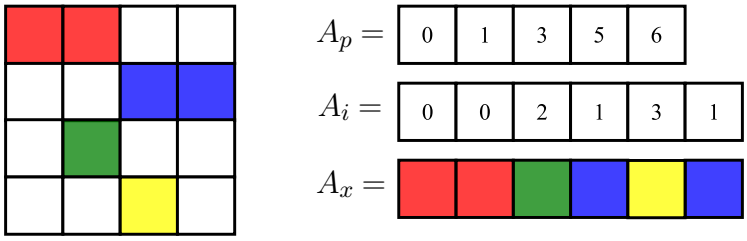


Figure 3.3: Matrix represented in the CSC format.

3.2.3 ELLPack Format

For an $M \times N$ matrix with a maximum of K non-zeros per row, the ELLPack format stores the non-zeros in an $M \times K$ matrix `data`, and an $M \times K$ matrix `indices`. The `data` matrix store the values of the non-zeros, and `indices` store the column index of every element. Rows that have fewer than K non-zeros are padded with zeros. Adapted from [15].

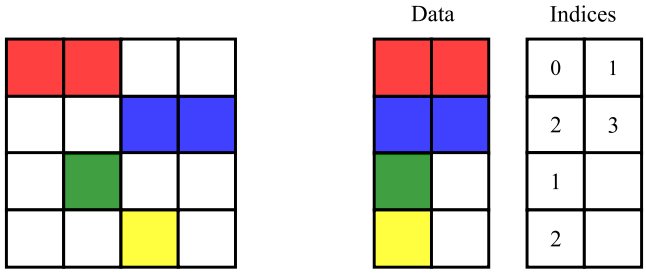


Figure 3.4: Matrix Represented in the ELLPACK format.

3.3 Sequential SpMV

A sequential implementation of SpMV on a matrix stored in the CSR format can be implemented in the following manner:

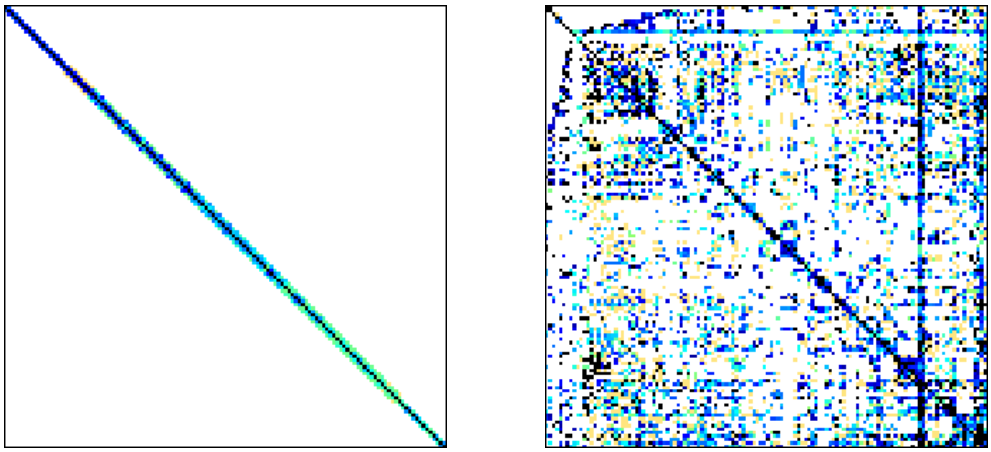
Algorithm 1: Sequential CSR-based SpMV

Input : A_p, A_j, A_x, x
Output : y

for $i \leftarrow 0$ **to** n **do**
  sum $\leftarrow 0$
  for $j \leftarrow A_p[i]$ **to** $A_p[i+1]$ **do**
    sum $=$ sum $+ A_x[j] \cdot x[A_j[j]]$
  $y[i] \leftarrow$ sum

For SpMV on well structured matrices such as those similar to the matrix illustrated in Figure 3.5, each non-zero element typically incurs a data movement cost of approximately 12 bytes and results in 2 floating-point operations (FLOPs). This estimation may appear inconsistent with the data access pattern described in algorithm 3.3, where two doubles and one integer are read per nonzero, corresponding to a total of 20 bytes. This discrepancy stems from the caching behaviours of the input vector x, as after an entry is initially read, it is frequently reused, and thus remains in cache. This means that on average, only one double and one integer is read per nonzero, which corresponds to 12 bytes.

In contrast, for highly unstructured matrices, the absence of data locality can significantly degrade cache reuse. In the worst case scenario, each non-zero may trigger the loading of an entire 64 byte cache line, with only a small fraction of it being used. Consequently, the effective data movement can increase to as much as 76 bytes per 2 FLOPs.



(a) Cube_Coup_dt0 (b) shermanACd

Figure 3.5: Well structured (a) and poorly structured (b) matrices.

3.4 Shared Memory SpMV

SpMV can be parallelized using the OpenMP directive `#pragma omp parallel for`. By default, this tells OpenMP to use `static` scheduling when parallelizing the outer iteration loop. When static scheduling is used, the span of the iteration that each thread will execute is precomputed, and stays static, as the naming suggests. There are other scheduling options, such as `dynamic` and `guided`, which will be discussed in later sections.

An implementation of shared memory SpMV is outlined below.

Algorithm 2: Shared Memory CSR-based SpMV

Input : A_p, A_j, A_x, x
Output : y

```
#pragma omp parallel for
for i ← 0 to n do
    sum ← 0
    for j ← A_p[i] to A_p[i+1] do
        sum = sum + A_x[j] · x[A_j[j]]
    y[i] ← sum
```

3.4.1 Scheduling options

As seen in algorithm 2, the outer loop is parallelized, which translates to dividing the rows of the matrix evenly among the threads. This work fine for well structured matrices, but for matrices with dense rows, such as the matrix shown in Figure 3.6, we obtain large imbalances in the computational load for each thread, which impacts performance. This is because the next iteration of SpMV cannot start until all threads have finished their computations on the current iteration, which means that threads will be idle while waiting for the slowest thread to finish its workload.

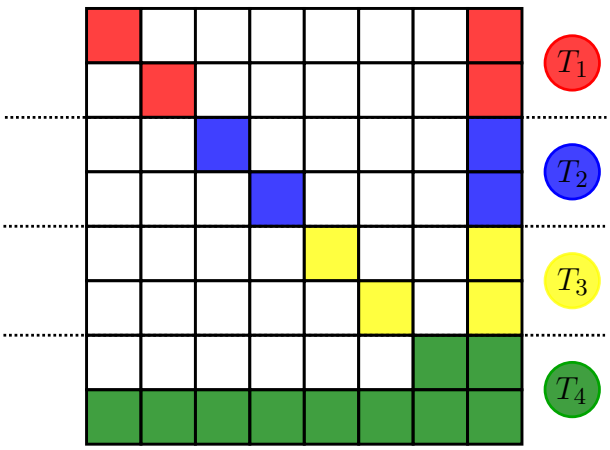


Figure 3.6: Row distribution among threads under static scheduling.

3.4.2 Dynamic Scheduling

Dynamic scheduling in OpenMP involves precomputing the range of iterations without assigning specific iteration subsets to individual threads in advance. Under static scheduling, if thread A receives sparse rows and thread B receives dense rows, thread A will become idle prematurely while thread B remains computationally engaged. In contrast, dynamic scheduling allocates iterations at runtime to threads as they become available, thus potentially balancing the computational load more effectively, particularly when iteration workloads vary significantly.

At first glance, dynamic scheduling appears to resolve the workload imbalance inherent in static scheduling, even though it comes with more overhead. While this assumption holds true on smaller systems, such as personal laptops equipped with fewer physical cores and a single processor socket, it does not readily apply to larger, dual-socket systems utilized for the experiments conducted in this thesis. Here, the first-touch memory policy becomes particularly significant.

3.4.3 Performance Implications of the First-Touch Policy

As shown in Table 2.1, accesses to main memory are significantly slower than accesses to local cache levels such as L1 or L2. On NUMA architectures, this disparity is further amplified when a thread must access memory located on a remote NUMA node. In dual-socket configurations, for example, such accesses require communication across the inter-socket interconnect. Using dynamic or guided scheduling, threads can be assigned iterations that requires memory localized on a different NUMA node, which in turn introduces additional latency and can lead to congestion of the memory bandwidth through the interconnect.

3.5 Distributed Memory SpMV

In distributed memory systems, the computational workload is distributed across multiple nodes, each typically comprising a dual-socket architecture with local memory. Unlike shared memory systems, data access across nodes is non-trivial and requires explicit communication, most commonly implemented using the Message Passing Interface (MPI).

For distributed sparse matrix-vector multiplication (SpMV), the computation proceeds similarly to the shared memory case (algorithm 2). The

sparse matrix is divided across nodes, and each node computes its local segment of the output vector y. At the end of each iteration, partial results are assembled to produce the global vector, necessitating inter-node communication.

While the thread scheduling and memory locality issues discussed in 3.4.1 remain relevant, distributed memory systems introduce the additional challenge of communication overhead. As shown in Table 2.1, inter-node communication is significantly more expensive than memory accesses. Consequently, minimizing the total communication volume per SpMV iteration is critical for performance.

3.5.1 Load Balancing and Graph Partitioning

Even with a well-balanced distribution, excessive communication between nodes can become a performance bottleneck if data dependencies span many partitions. An effective partitioning strategy must therefore strike a careful balance between distributing the computational load evenly and reducing the amount of communication required between ranks. This trade-off is naturally captured by the graph partitioning problem.

The graph partitioning problem involes dividing an undirected graph $G(V, E)$, where V is the set of vertices and E is the set of edges into K disjoint subsets (or partitions) such that each subset contains a comparable amount of vertices, and the number of edges connecting different subsets is minimized. Each vertex $v_i \in V$ may be assigned a weight w_i, and each edge $e_{ij} \in E$ may carry a cost c_{ij}. The sum of weights W_k of each partition P_k must not exceed a specified imbalance threshold ϵ relative to the average weight of each partition. An edge can be classified as either an internal edge (those that connect nodes in the same partition), or an external edge (those that connect vertices in different partitions), and the cutsize is defined as the sum of the costs of all external edges. The goal is to minize the cutsize, while preserving the balance between the size of each partition. This problem is known to be NP-Hard, even for obtaining bipartitions on unweighted graphs (adapted from [2]).

Algorithms that aim to solve the graph partitioning problem rely on heuristics that provide good approximations in practice. Optimizing for one objective at the expense of the other is trivial but undesirable: assigning all vertices to a single partition minimizes communication to zero but results in extreme load imbalance, whereas naïvely assigning vertices to maintain balance without regard to connectivity may lead to

high communication costs.

3.5.2 METIS

The graph partitioner used for the experiments in this thesis is called METIS. METIS is a software package for partitioning large irregular graphs, and has other uses such as partitioning meshes and computing fill-reducing orderings of sparse matrices. METIS provides algorithms that are based on multilevel graph partitoning. These algorithms partition the graph through a coarsening and refinement phase. In the coarsening phase, the size of the graph is reduced by way of collapsing the vertices and edges, and then partition the smaller graph. In the refinement phase, the graph is gradually expanded to its original size, and the portion of the graph that is close to the boundary is the primary focus, ensuring the quality of the partition is kept (adapted from [6]). algorithm 3 shows how to partition and reorder the vertices of a graph according to the generated partition, using METIS' K-way partition algorithm.

The way METIS partitions graph does not necessarily optimize for minimizing the communication volume, but rather obtaining partitions that are roughly equal in size. It is possible that there are partitions that have a larger imbalance between the size of partitions, but much smaller communication volumes. In order to obtain partitions that optimize for minimizing communication, it is necessary to look to hypergraph partitioners.

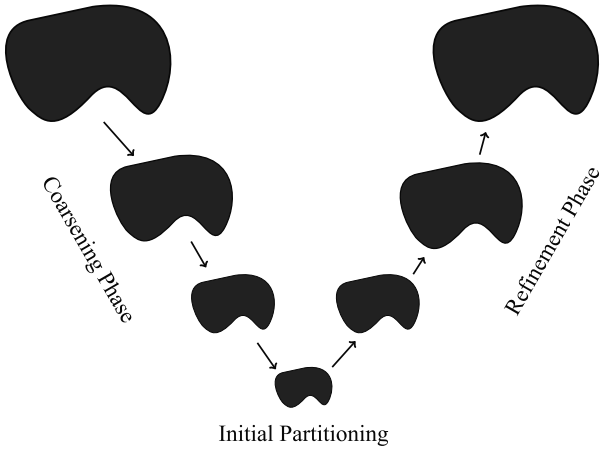


Figure 3.7: Illustration of multilevel graph partitioning (adapted from [6]).

3.5.3 Hypergraph Partitioning

> A *hypergraph* $H = (V, N)$ is defined as a set of vertices V and a set of nets (hyperedges) N among those vertices. Every net $n_j \in N$ is a subset of vertcies, i.e., $n_j \subseteq V$. The vertices in a net n_j are called its *pins* and denoted as $pins[n_j]$. The size of a net is equal to the number of its pins, i.e., $s_j = |pins[n_j]|$. The set of nets connected to a vertx v_i is denoted as $nets[v_i]$. The degree of a vertex is equal to the number of nets it is connected to, i.e., $d_i = |nets[v_i]|$.
>
> [2]

Hypergraph partitioners try to solve the same problems, and employ similar algorithms as those graph partitioners use, however they use these algorithms on hypergraphs, and not regular graphs. Çatalyürek and Ayakanat showed in [2] that using the hypergraph partitioner they developed, they could get up to 63% less communication volume, with an average reduction in communication volume of 30% - 38%, when compared to METIS.

As the improvement of the communication strategies are the main focus of the contents of this thesis, the specific partitioner tool that is used is not the biggest concern, and the programs implemented have

opted for the usage of METIS, which as mentioned is a regular graph partitioner. It would be possible to refactor the code in such a way that a hypergraph partitioner such as PaToH can be used.

Algorithm 3: Partitioning and reordering a matrix.

Input : g, n_p, p
Output : Partitioned and reordered g

if $n_p = 1$ **then**
    p[0] ← 0
    p[1] ← $g.n_r$
    return g
partitionVector ← METIS_PartGraphKway(`arguments specifying constraints for partition`)
newId ← $[0] \cdot g.n_r$
oldId ← $[0] \cdot g.n_r$
id ← 0
p[0] ← 0
for $r \in \{0, \ldots r\}$ **do**
    for $i \in \{0, \ldots, g.n_r\}$ **do**
        if *partitionVector[i]* $= r$ **then**
            oldId[id] ← i
            newId[i] ← id
            id ← id + 1
    partitionVector[$r+1$] ← id
newRowPtr ← $[0] \cdot g.n_r + 1$
newColIdx ← $[0] \cdot g.n_c$
newValues ← $[0] \cdot g.n_c$
for $i \in \{0, \ldots, g.n_r - 1\}$ **do**
    d ← g.rowPtr[oldId[i]+1] - g.rowPtr[oldId[i]]
    newRowPtr[i + 1] ← newRowPtr[i] + d
    for $j \in \{0, \ldots, d-1\}$ **do**
        newColIdx[newRowPtr[i] + j] ← g.colIdx[g.rowPtr[oldId[i]] + j]
        newValues[newRowPtr[i] + j] ← g.values[g.rowPtr[oldId[i]] + j]
    for $j \in \{newRowPtr[i], \ldots, newRowPtr[i+1]-1\}$ **do**
        newColIdx[j] ← newId[newColIdx[j]]
g.rowPtr ← newRowPtr
g.colIdx ← newColIdx
g.values ← newValues
return g

3.6 SuiteSparse Matrix Collection

The matrices that have been used for SpMV in this thesis all come from the SuiteSparse Matric Collection. This is a large dataset of sparse matrices that come from real world applications. The matrices available here cover problems within structural engineering, CFD, FEM, thermodynamics, optimization, economic and financial modeling, amongst many more. For more information on the SuiteSparse Matrix Collection, see [3].

Chapter 4

Communication Strategies

This chapter introduces the five different communication strategies that have been tested in the experiments conducted for this thesis. Each communication strategy implements increasingly more selective ways of communicating the data-dependencies that arise when the matrix is partitioned amongst the MPI ranks. Starting from the most simple strategy, where the entire input vector x is communicated for each iteration of SpMV, moving on to two separator based methods, and finally two strategies where only the required elements that incur data-dependencies are being communicated.

In the subsequent sections, each figure illustrating a communication strategy is based on the graph shown in Figure 4.1. In this figure each color represents a partition, and the nodes residing within each color belongs to that partition.

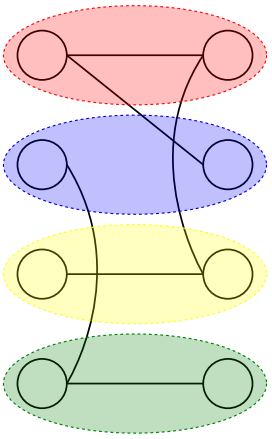


Figure 4.1: Simple graph partitioned amongst ranks - each color represents a rank.

4.1 Strategy A - Exchange entire vector

In this strategy, at each iteration every rank sends its locally computed values of y to all other ranks so that each process obtains a complete copy of the output vector before proceeding. This approach can be implemented with the MPI collective operation `MPI_Allgatherv`, which supports different send and receive counts on each rank. 4 shows how this communication pattern can be implemented.

Algorithm 4: Strategy A - Communication Pattern

```
for each iteration do
    spmv(g,x,y)
    MPI_Allgatherv(local_y, sendcount, MPI_DOUBLE, y,
     recvcounts, displs, MPI_DOUBLE, MPI_COMM_WORLD)
    swap pointers of x and y
```

Let n_p denote the total number of ranks, and let $|x_i|$ be the number of vector entries assigned to rank i (given by the average; $\frac{|x|}{n_p}$). Since each rank must send its local block size of $|x_i|$ to the other $n_p - 1$ ranks, the total volume of communication per iteration is given by 4.1.

$$n_p \cdot (n_p - 1) \cdot |x_i| \tag{4.1}$$

Figure 4.2 illustrates the state of the vector before and after communication using this strategy.

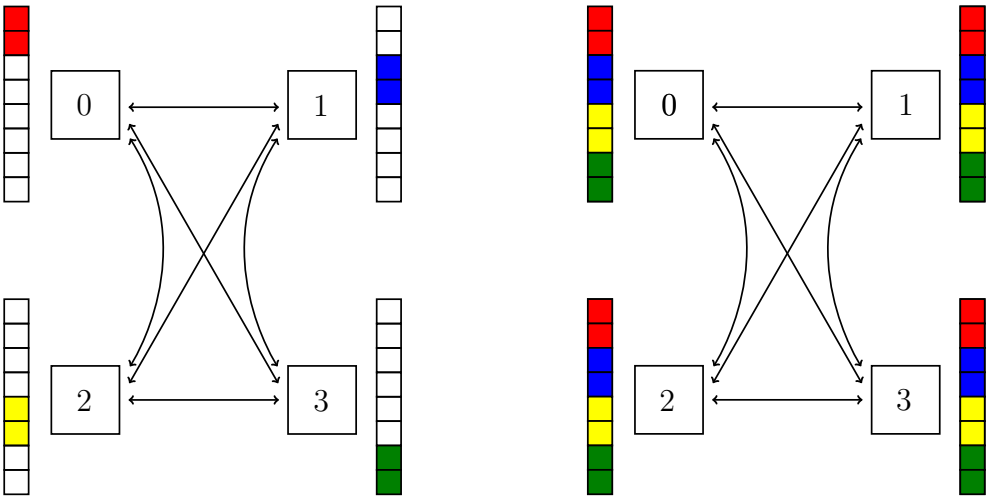


Figure 4.2: Contents of each ranks vector before and after communication under the *Exchange Entire Vector* communication pattern.

4.2 Strategy B - Exchange only separators

The next communication strategy improves upon strategy A by recognizing that only the set of each partitions elements of x that induce data-dependencies require explicit communication. The set of these elements is called a separator. This communication strategy is at least as good as strategy A regardless of the matrix structure. The size of any partitions separator can at most be as large as the set of elements of that partition, but is usually smarter, thus reducing the communication volume.

Let n_p denote the total number of ranks, and let s_i be the size of the separator of rank i. Since each rank sends its separator to each other rank (i.e. $n_p - 1 other ranks$), then the total communication volume is given by 4.2.

$$\sum_{i=1}^{n_p} (n_p - 1) \cdot s_i \tag{4.2}$$

To facilitate this strategy, separator values are reordered such that they appear at the beginning of each process's local segment of x. Once this structure is established, communication is performed using `MPI_Allgatherv`,

transmitting only the subset of x that contains separator values. The number of separators on each process must be known beforehand, which can be computed by counting the number of elements that have neighbours belonging to a different partition.

4.2.1 Reordering

After partitioning the matrix into different parts, we obtain a partition vector p, where the $p[i]$ stores the index of the partition the i^{th} entry in A_p. It is necessary to reorder the entries in A_p in accordance with the partition vector, such that all entries belonging to the same partition are stored in sequence. The algorithm below gives an outline of how this can be achieved. Here n_p is the number of partitions, n_r is the size of A_p, and n_c is the size of A_j and A_x.

Algorithm 5: Reordering of Separators

Input : $n_p, n_r, n_c, p, A_p, A_j, A_x$
Output : Reordered A_p, A_j, A_x

```
newId ← [0] · n_r
oldId ← [0] · n_r
id ← 0
p_0 ← 0

for r ∈ {0, ..., n_p − 1} do
    for i ∈ {0, ..., n_r − 1} do
        if p[i] = r then
            oldId[id] ← i
            newId[i] ← id
            id ← id + 1
    p[r + 1] ← id

newV ← [0] · (n_r + 1)
newE ← [0] · n_c
newA ← [0] · n_c

for i ∈ {0, ..., n_r − 1} do
    degree ← A_p[oldId[i] + 1] − A_p[oldId[i]]
    newV[i + 1] ← newV[i] + degree

for i ∈ {0, ..., n_r − 1} do
    degree ← A_p[oldId[i] + 1] − A_p[oldId[i]]
    for j ∈ {0, ..., degree − 1} do
        newE[newV[i]+j] ← A_j[A_p[oldId[i]] + j]
        newA[newV[i]+j] ← A_x[A_p[oldId[i]] + j]
    for j ∈ {newV[i], ..., newV[i + 1] − 1} do
        newE[j] ← newId[newE[j]]

Overwrite A_p ← newV, A_j ← newE, A_x ← newA
```

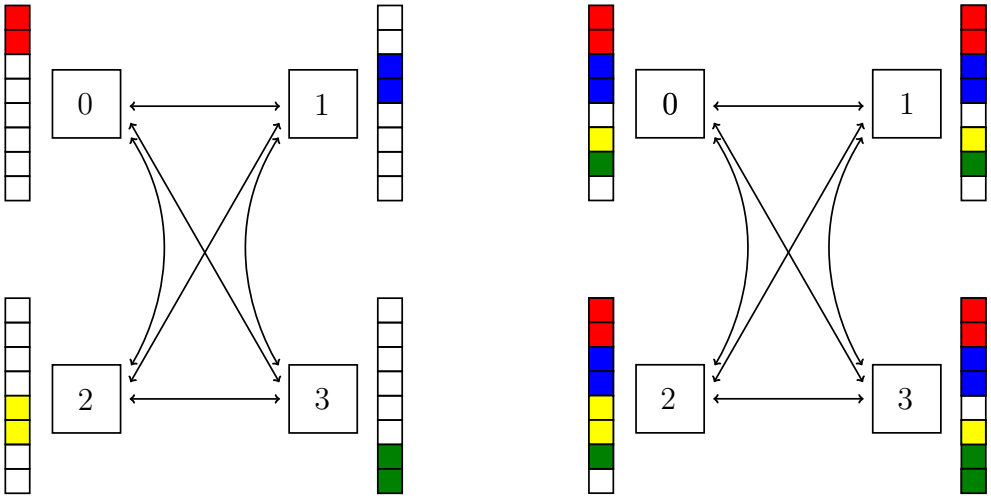


Figure 4.3: Contents of each ranks x vector before and after communication under the *Exchange Separators* communication pattern.

4.3 Strategy C - Exchange only required separators

Further reduciton to the communication volume can be achieved by observing that not all separator values are required by every process. As the number of partitions increases, the set of dependencies between partitions tends towards sparcity. As a consequence of this, certain sets of separators may only need to be communicated to a given subset of processes. Using this strategy, each process only communicates its set of separator values to the processes that require them. The communication pattern in illustrated in Figure 4.4, and Algorithm 6 shows how this can be implemented.

Let n_p be the total number of ranks, n_i be the number of neighbours of rank i, and s_i the size of the separator of rank i. With $n_i \leq n_p$, the total communication volume is given by 4.3.

$$\sum_{i=1}^{n_p} s_i \cdot n_i \tag{4.3}$$

Algorithm 6: Exchange only required separators

Input : y, rank,size, displacements, sendItems, sendCount
Output :

MPI_Requests ← 0
for *r = 0; r < size; r++* **do**
  if *rank = r or sendItems[r][rank] = 0* **then**
    continue
  Post non-blocking MPI receive for $sendCount[r]$ elements at address $y + displacements[r]$
  MPI_Requests++
for *r = 0; r < size; r++* **do**
  if *rank = r or sendItems[r][rank] = 0* **then**
    continue
  Post non-blocking MPI send of $sendCount[rank]$ elements to address $y + displacements[rank]$
  MPI_Requests++
Wait for all non-blocking requests to complete

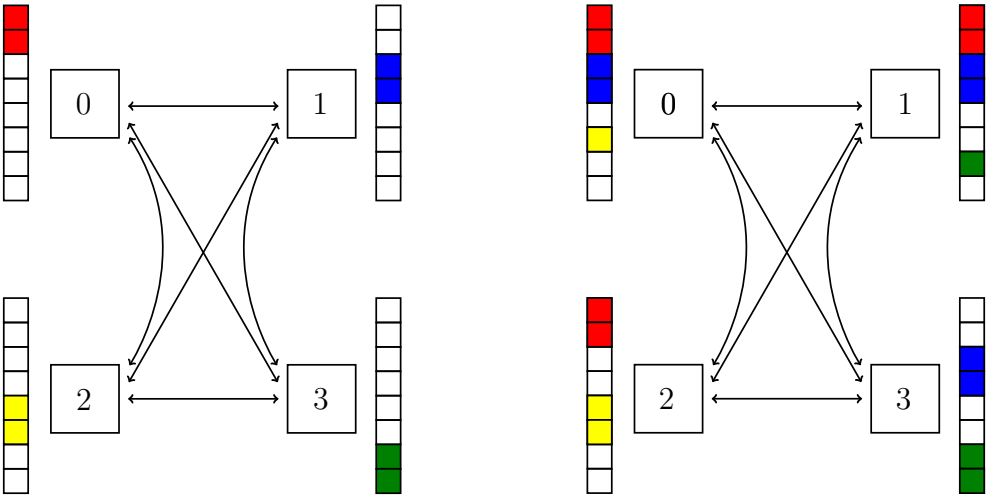


Figure 4.4: Contents of each ranks x vector before and after communication under the *Exchange Required Separators* communication pattern.

4.4 Strategy D - Exchange only required separator values

The final strategy further reduces the communication volume by only communicating the separator elements to the rank with which that element induces a data dependency. This eliminates unnecessary communicaton, but introduces complexity in managing said communication. It requires careful management of the send and recieve lists, and introduces some additional overhead by means of a packing and unpacking step necessary for transmitting the data between ranks. This overhead does however not outweigh the benefits gained from the reduction in communication volume which this strategy yields.

Algorithm 7 shows how the communication step for this strategy is implemented. Both strategy B and C requires reordering of the separator values, in order to make the communication of the separators easier. In contrast, strategy D does not require reorderding of the separators, as it packs each of the separators into send lists before communication. The original index of the separator element is stored, and is used to unpack it into the correct position of x after communication.

Algorithm 7: Strategy D - Packing, exchanging and unpacking separator elements

```
Input  : c, x, rank, size
Output : Updated vector x
totalSend, totalRecv ← 0
for i = 0; i < size; i++ do
    totalSend ← totalSend + c.sendCount[i]
    totalRecv ← totalRecv + c.receiveCount[i]
Allocate sendBuffer[totalSend], recvBuffer[totalRecv]
sendOffset ← 0
for i = 0; i < size; i++ do
    for j = 0; j < c.sendCount[i]; j++ do
        sendBuffer[sendOffset++] ← x[c.sendItems[i][j]]
Compute sendDispls, recvDispls as prefix sums of c.sendCount,
 c.receiveCount
MPI_Ialltoallv(sendBuffer, c.sendCount, sendDispls, recvBuffer,
 c.receiveCount, recvDispls)
MPI_Waitall()
recvOffset ← 0
for i = 0; i < size; i++ do
    for j = 0; j < c.receiveCount[i]; j++ do
        x[c.receiveItems[i][j]] ← recvBuffer[recvOffset++]
Free sendBuffer, recvBuffer, sendDispls, recvDispls
```

Let n_p be the total number of ranks, and let d_i be the number of elements in rank is separator that are required by other ranks. Since this communication strategy only communicated these elements, the total communication volume is given by 4.4.

$$\sum_{i=1}^{n_p} d_i \tag{4.4}$$

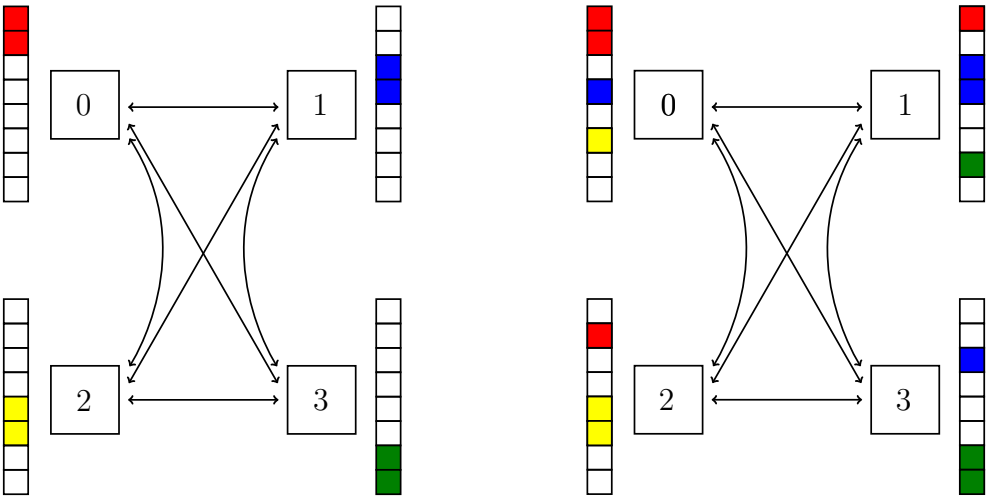


Figure 4.5: Contents of each ranks x vector before and after communication under the *Exchange Required Elements* communication pattern.

4.5 Strategy E - Memory Scalable

The communication strategies discussed so far all have a common problem that prevents them from scaling to large matrices. These strategies all store the entire vector x, and will run into performance issues when x is so large that it doesn't fit into memory. Usually, this is not a problem when SpMV is ran on CPUs, as they have large amounts of memory. Even on GPUs this problem might not be encountered, as modern GPUs have sufficient memory for large matrices.

Instead of storing the entire vector, each rank only stores its local part of the vector. In addition, it is necessary to allocate enough space for the separators elements that are needed from the other ranks. In order to achieve this, x is renumbered such that every ranks part of the vector is 0-indexed. For the local part of the x vector this is done by simply subtracting the position of the first local entry assigned to that rank from each entry in the local vector. It is also necessary to reorder the separator elements, as they will be stored after the entries in the local vector. This is achieved in a similar manner as the algorithm outlined in 5.

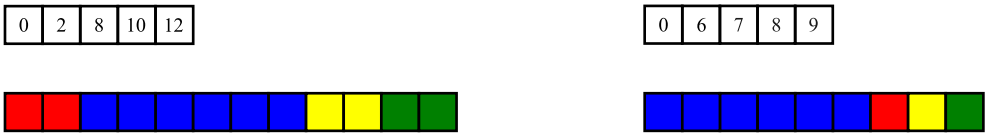


Figure 4.6: Global indexing vs. Local indexing of x.

Table 4.1 gives an overview of the reductions in communication strategies described in the previous sections. As the graph in Figure 4.1 is very small, this table doesn't illustrate the full potential of any of the strategies, but it does show that the communication volume is reduced for each strategy.

Strategy	**% of x communicated**	**% diff. from prev. strategy**
A	100	N/A
B	75	25
C	41.7	33.3
D	25	16.7

Table 4.1: Overview of communication volume reduction for strategy A-D using graph in Figure 4.1.

Chapter 5

Results

5.1 Theoretical Maximum

As has been established, the maximum performance scales with the available memory bandwidth of the system, and not with the amount of processes used. For SpMV, a total of 6 bytes are read per FLOP, which means that the theoretical maximum is given by 5.1. This is under the assumption that all resources on the system is dedicated to the SpMV operation, which is not the case in reality. At most, somewhere in the neighbourhood of 80% of the systems resources can be expected to be utilized.

$$\text{Maximum Theoretical Performance} = \frac{\text{Memory bandwidth}}{6} \tag{5.1}$$

5.2 Experimental Setup

In order to rigorously evaluate and compare the performance of the five communication strategies, a systematic series of experiments were ran on the eX^3 machine. Each experiment executes 100 iterations of SpMV on a given sparse matrix, with a given configuration and a given communication strategy. Every program used has been compiled GCC 11.4.0 using the compiler flags `-march=native` and `-O3`.

5.2.1 Hardware Platforms

The experiments that have been ran target three high-performance computing systems. Table 5.1 gives an overview of the hardware that has been used in the experiments.

	FPGAQ	Rome	Defq
CPUs	AMD EPYC 7413	AMD EPYC 7302P	AMD EPYC 7601
Instr. setu w	x86-64	x86-64	x86-64
Microarch.	Zen 3	Zen 2	Zen 1
Sockets	2	1	2
Cores	2×24	1×16	2×32
Freq. [GHz]	2.6–3.6	1.5–3.3	2.2–3.2
L1I/core [KiB]	64	32	64
L1D/core [KiB]	32	32	32
L2/core [KiB]	512	512	512
L3/socket [MiB]	128	16	128
Mem. channels	2×8	1×8	2×8
Bandwidth [GB/s]	N/A	204.8	N/A
Triad [GB/s]	N/A	90.9	N/A

Table 5.1: Hardware used in the experiments. STREAM Triad was run with the `-march=native` and `-O3` compilation flags.

There has been performed experiments measuring both single node and multi node performance. For single node experiments, the number of MPI ranks is doubled until all cores on the given node is assigned a MPI rank, also OpenMP was not utilized for these experiments.

For multi node experiments, two configuration have been tested. The first configuration involves assinging one MPI rank per node, and utilizing shared memory paralleization through the use of OpenMP within each node. The second configuration places one MPI rank per socket. These experiments aims to show the performance of the communication strategies on multi node systems. Furthermore, it is possible to compare the impact that the number of memory controllers utilized have on the results. Using one MPI rank per node only utilizes one of two memory controllers on a dual socketed systems, whereas using one MPI rank per socket utilizes both of the memory controller on the node.

5.2.2 Communication Strategies

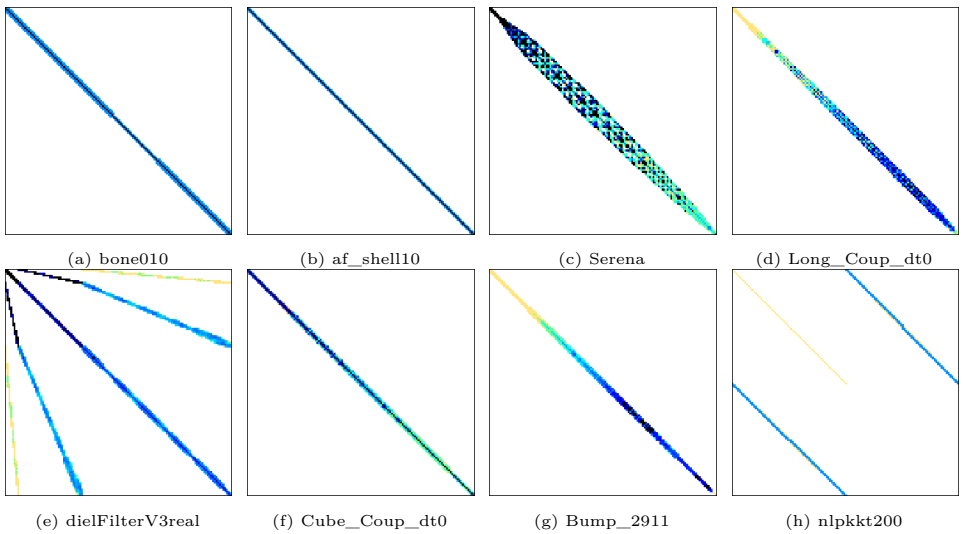
The communication strategies used have been extensively discussed in 4, and Table 5.2 gives an overview of what the various communication strategies will be referred to in the folliwing sections, and the key idea they use to communicate the data-dependencies between ranks.

Table 5.2: Names and description of communication strategies that have been tested.

Strategy name	**Strategy Description**
Strategy A	Exchanges entire local vector.
Strategy B	Exchanges entire separator.
Strategy C	Exchanges required separator.
Strategy D	Exchanges required separator elements.
Strategy E	Exchanges required separator elements, memory scalable.

5.2.3 Matrices

As the field of interest is that of results from scientific computing, the matrices shown in Figure 5.1 have been selected, and Table 5.3 gives a brief description of the problem the matrices arise from. Matrices that are based on scientific computing problems are generally well structured and have nonzero entries centered around the main diagonal. Matrices with these kinds of structures tend to yield a larger cache hit rate than matrices based on social networks and other kinds of graphs (see [14]). It is important to keep this in mind when interpreting the results, as other kinds of graphs can give very different results.



(a) bone010 (b) af_shell10 (c) Serena (d) Long_Coup_dt0

(e) dielFilterV3real (f) Cube_Coup_dt0 (g) Bump_2911 (h) nlpkkt200

Figure 5.1: Matrices used to generate results.

Name	Purpose
bone010	Trabecular Bone Micro-Finite Element Model
af_shell1	Sheet metal forming matrix
Serena	Gas reservoir simulation for CO_2 sequestration
Long_Coup_dt0	3D coupled consolidation problem (geological formation)
dielFilterV3real	High-order vector finite element method in EM
Cube_Coup_dt0	3D coupled consolidation problem (3D cube)
Bump_2911	3D geomechanical reservoir simulation
nlpkkt200	Symmetric indefinite KKT matrix

Table 5.3: Names and descriptions of the matrices used in the experiments.

5.3 AMD EPYC 7601

For experiments on the dual-socketed AMD EPYC 7601 nodes, three sets of experiments were conducted. The first set of experiments test the performance when using a single node without shared memory paralleization, The second set places only one MPI rank per node, and the final set of experiments place one MPI rank per socket. Both multi node experiments use shared memory parallelization for intra node communication.

5.3.1 Single Node Performance

For the single node experiments, no shared memory paralleization has been utilized. Each communication strategy has been ran using $1, 2, 4, 8, 16, 32$ and 64 MPI ranks, as the dual socketed system provides two 32-core AMD EPYC 7601 chips.

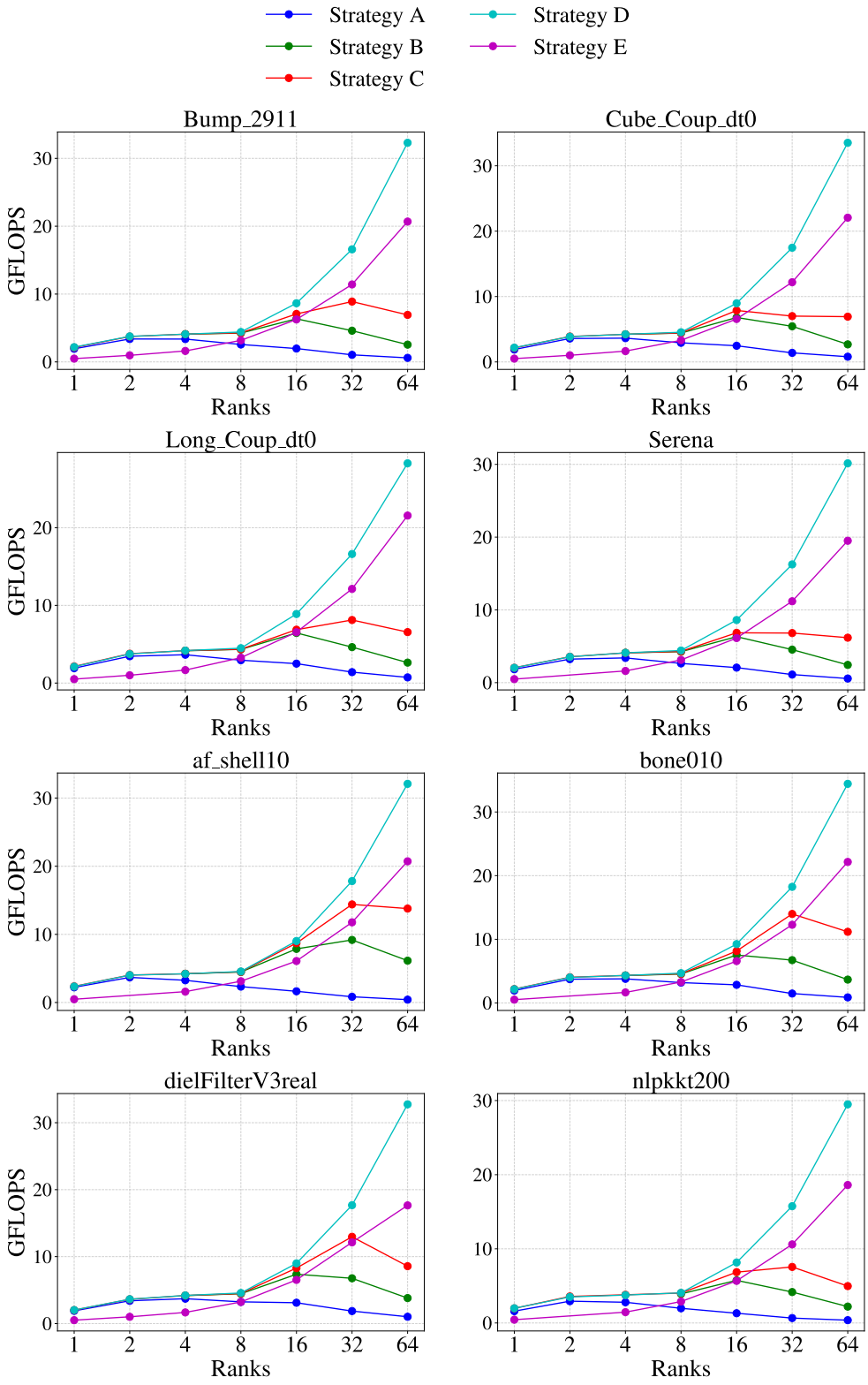


Figure 5.2: Sustained performance (GFLOPS) over 100 iterations of SpMV on a single node equipped with dual-socket AMD EPYC 7601 processors.

Figure 5.2 presents the achieved GFLOPS for each communication strategy. The performance trends largely align with expectations: each successive strategy generally exhibits improved performance over its predecessor. However, an exception arises with strategy E, which consistently underperforms relative to strategy D.

As discussed in section 4.5, strategy E reorders the local vector and separators. The effect this reordering has on well ordered matrices is that it can reduce cache reuse. This has not been measured directly, but it is the most likely culprit to the discrepancy between the results of strategy D and E.

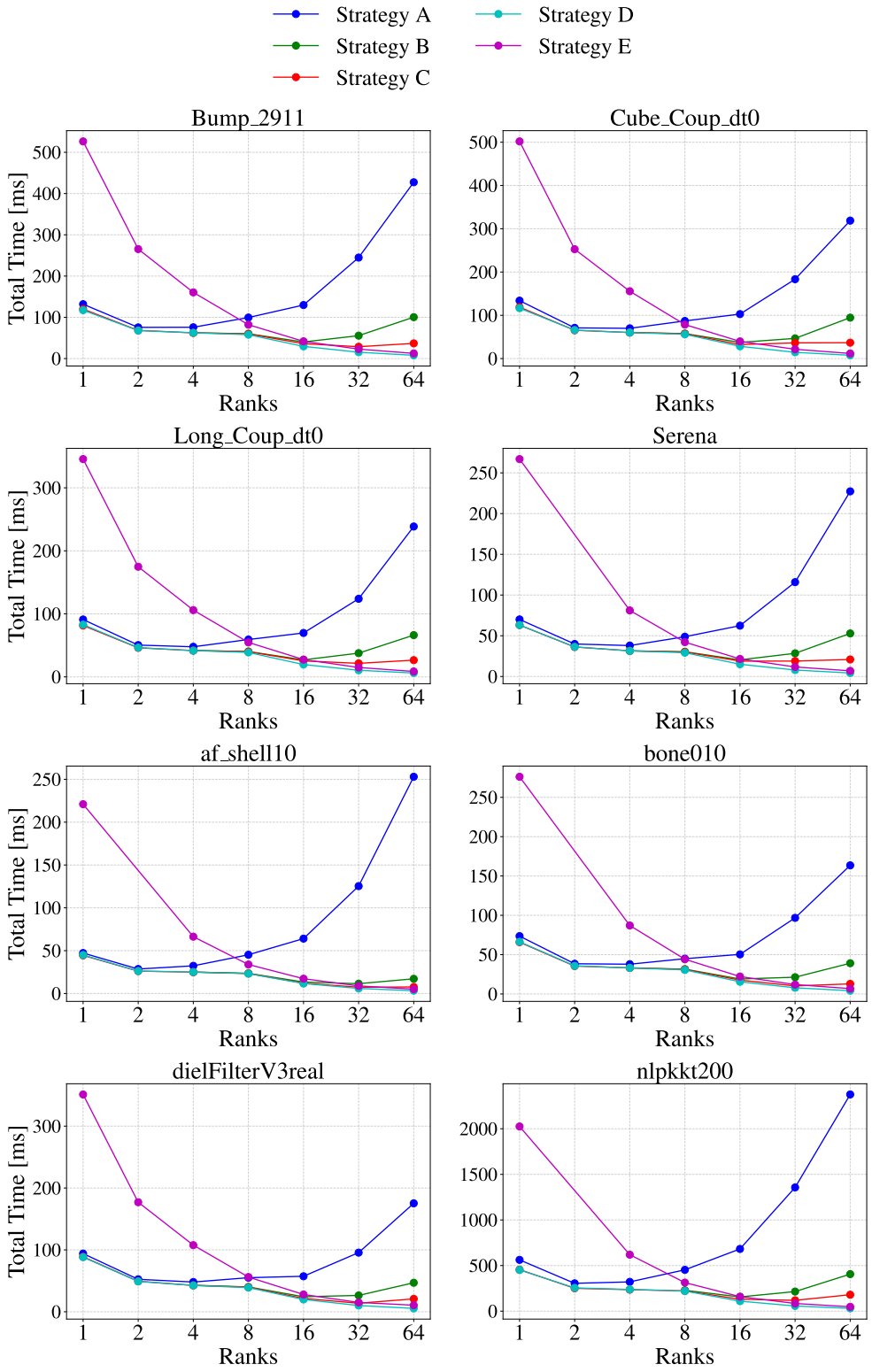


Figure 5.3: Total execution time of each communication strategy on a single node equipped with dual-socket AMD EPYC 7601 processors.

Figure 5.3 presents the total execution time for each communication strategy. It is closely related to Figure 5.2, as computing the GFLOPS number is given by the total number of flops performed divided by the total execution time.

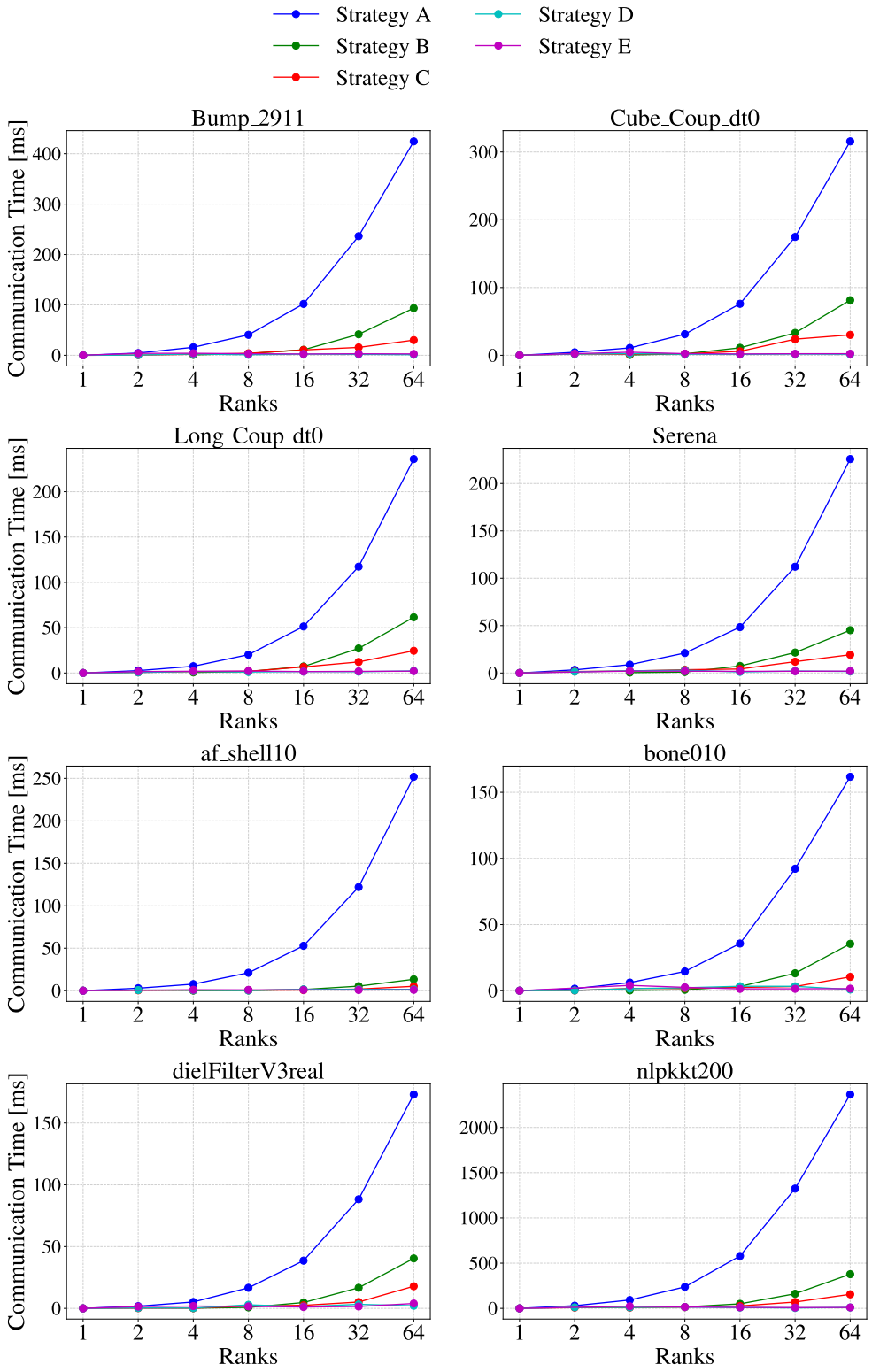


Figure 5.4: Communication time per iteraton of SpMV on a single node equipped with dual-socket AMD EPYC 7601 processors.

Figure 5.4 present the time spent communicating the data data-dependencies between MPI ranks for each iteration of SpMV. The results clearly illustrate the performance benefits of optimizing communication in distributed single node SpMV. Strategy A acts as a baseline, by naively communicating the entire vector. Each successive communication strategy implements measures to reduce the communication volume, and it can be seen that this does indeed result in lower communication times. Strategy B limited communication to communicating only separators, Strategy C narrowed this to only communicating the separators to the rank that need them. Finally, strategy D and E further reduces the communication volume by only communicating necessary separator elements. This progresion confirms the role communication strategies play in reducing the bottleneck that the data-dependencies between nodes induce. The results emphasize that careful attention to communication minimization yields improvements communication time, and by extension in performance for memory-bound operations like SpMV.

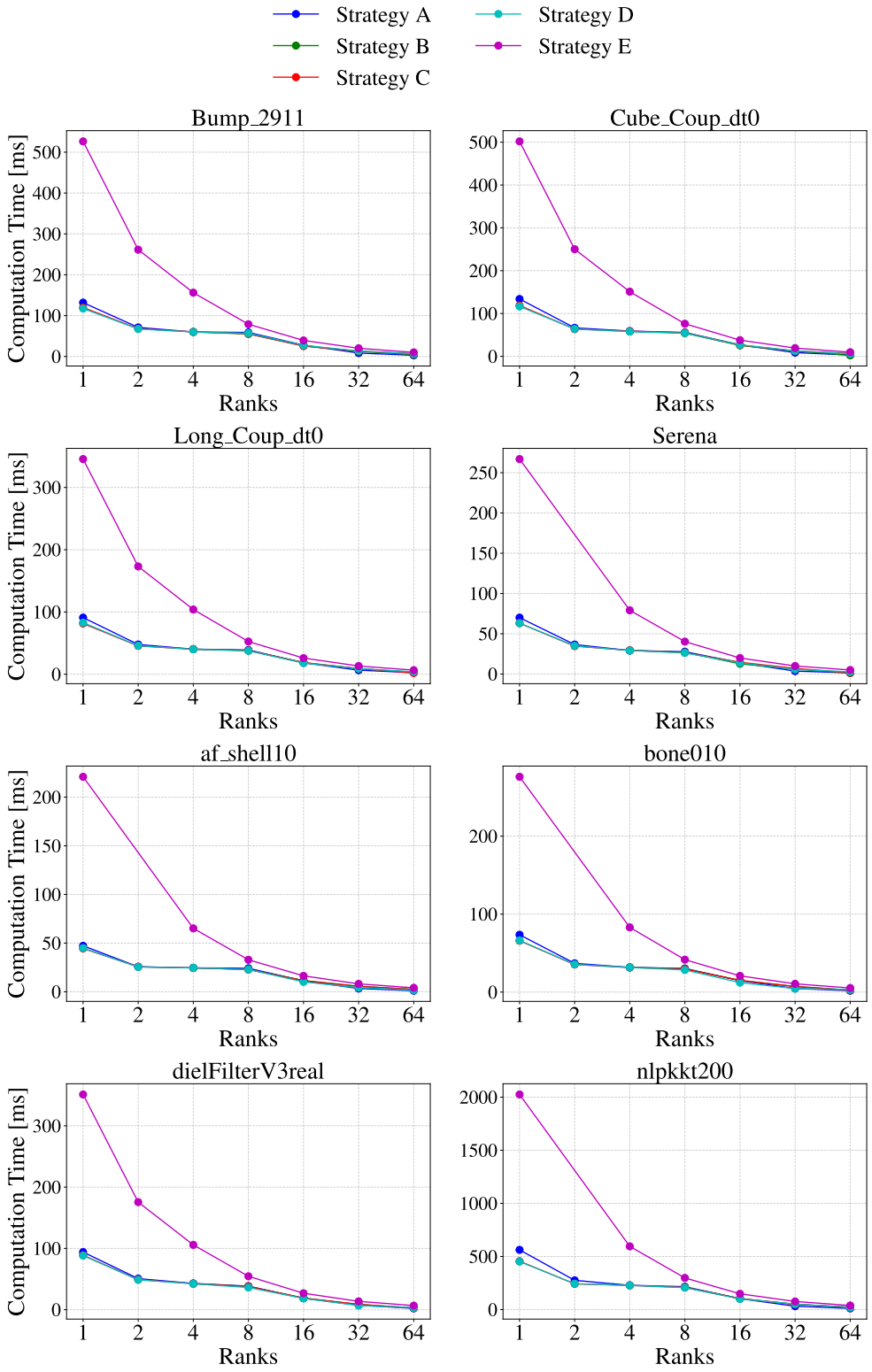


Figure 5.5: Computation time per iteraton of SpMV on a single node equipped with dual-socket AMD EPYC 7601 processors.

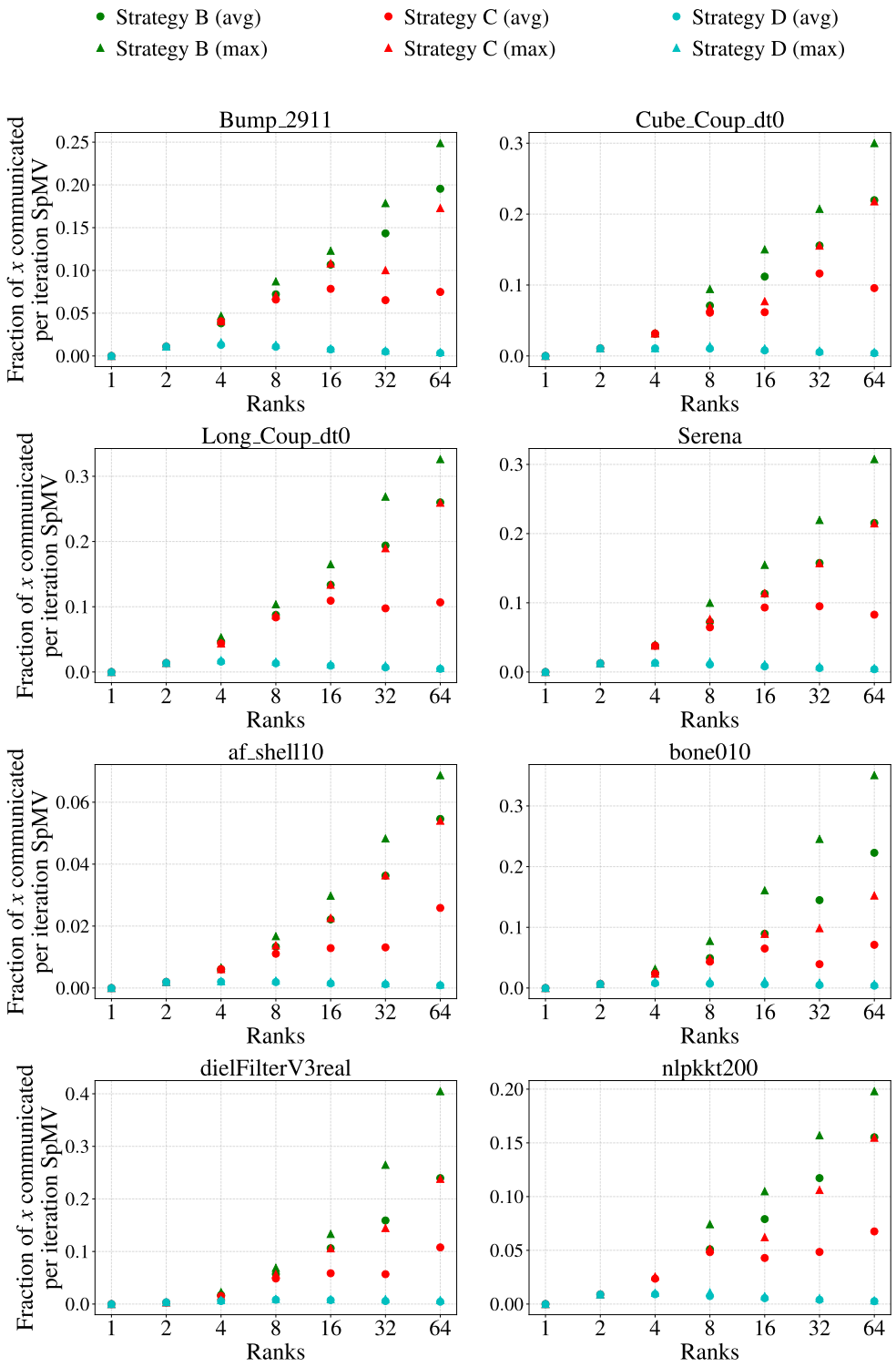


Figure 5.6: Fraction of the size of the global x vector communicated per SpMV iteration.

Figure 5.6 shows the fraction of the input vector x communicated per SpMV iteration across various matrices and ranks for different communication strategies. This figure provides an important quantitative counterpart to the timing data: it confirms that strategies which yielded lower communication time also effectively minimized the volume of communicated data. The results presented in this figure only show the communication volume for strategies B-D. Strategy A have been left out as the fraction of x communicated in each iteration is 1 no matter how many ranks are used. Strategy E has been left out as the communication volume for this strategy is the same as strategy D.

Notably, when comparing strategy B and C, the rank with the maximum communication volume in strategy C is usually equal to, or lower than the average communication volume of strategy B.

5.3.2 Multi Node Performance

The multi node experiments illustrate the performance across 1-4 dual-socketed nodes. Two configurations have been tested: One MPI rank per node with 64 OpenMP threads per rank, which utilizes only one of the two memory controller per node. Next, Two MPI ranks per node, or one MPI rank per socket, with 32 OpenMP threads per rank, which utilizes both memory controllers available per node.

One MPI rank per node

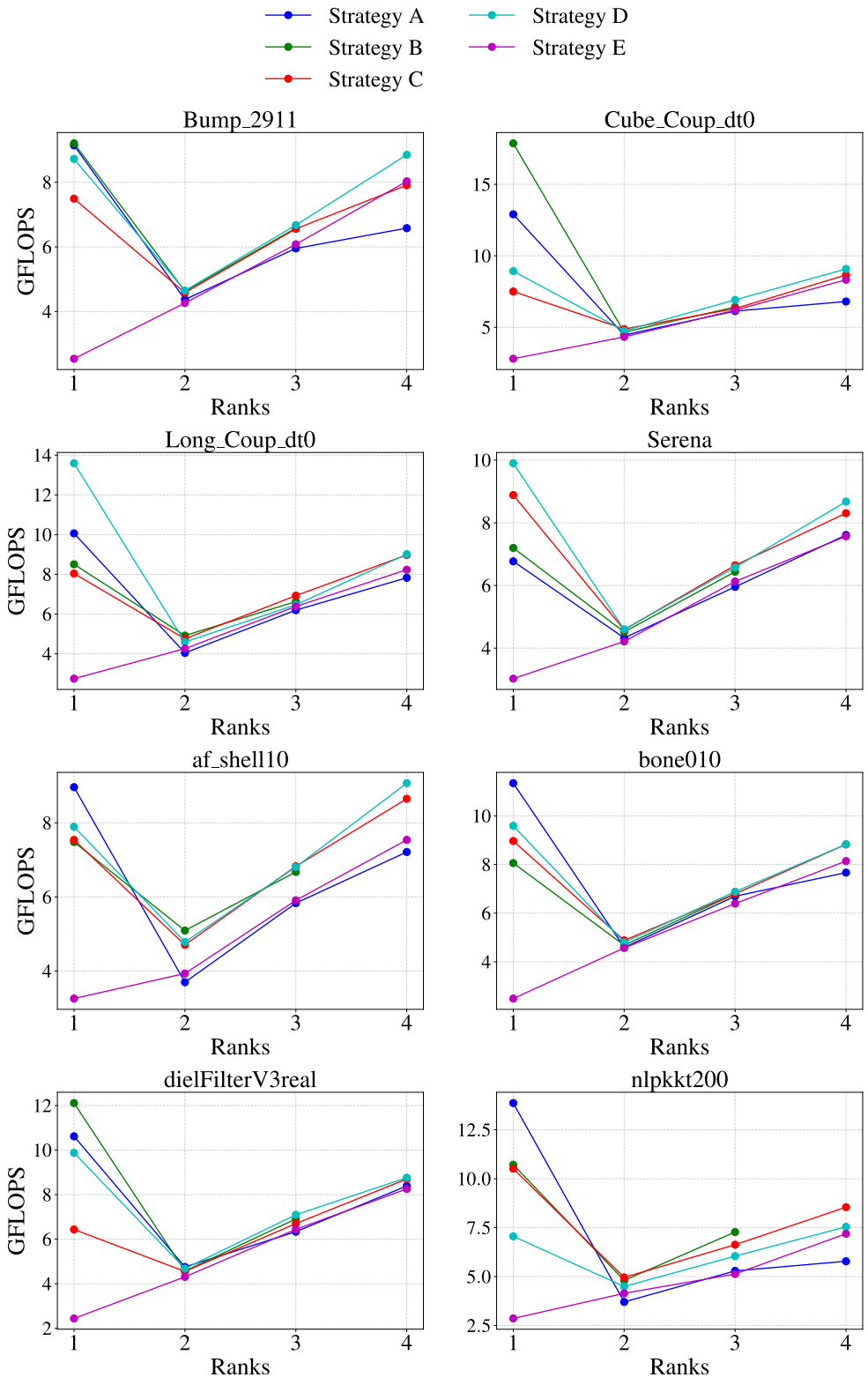


Figure 5.7: Sustained GFLOPS performance of SpMV over 100 iterations on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7601 processors.

The results illustrate some quirks that are inherent to the AMD EPYC 7601 chip. As is evident from the plots, using only one node (and one MPI rank), consistently yield results that are either better or almost as good as the results achieved when using the maximum amount of available nodes, and when compared to using 64 MPI ranks - one per physical core - the performance is much worse. As an example, for the `Bump_2911` matrix, strategy D achieves a GFLOPS performance of ≈ 9 GFLOPS on one node, with 64 OpenMP shared memory threads. When this is compared to Figure 5.2 using 64 MPI ranks, which achieves ≈ 30 GFLOPS for the same matrix.

It is clear that using only distributed memory far outperforms using only shared memory parallelization. The reasons for this is outside the scope of this thesis, but in brief, the AMD EPYC 7601 utilizes the first generation Zen microarchitecture, which had some problems in this area. (ADD REFERENCE HERE).

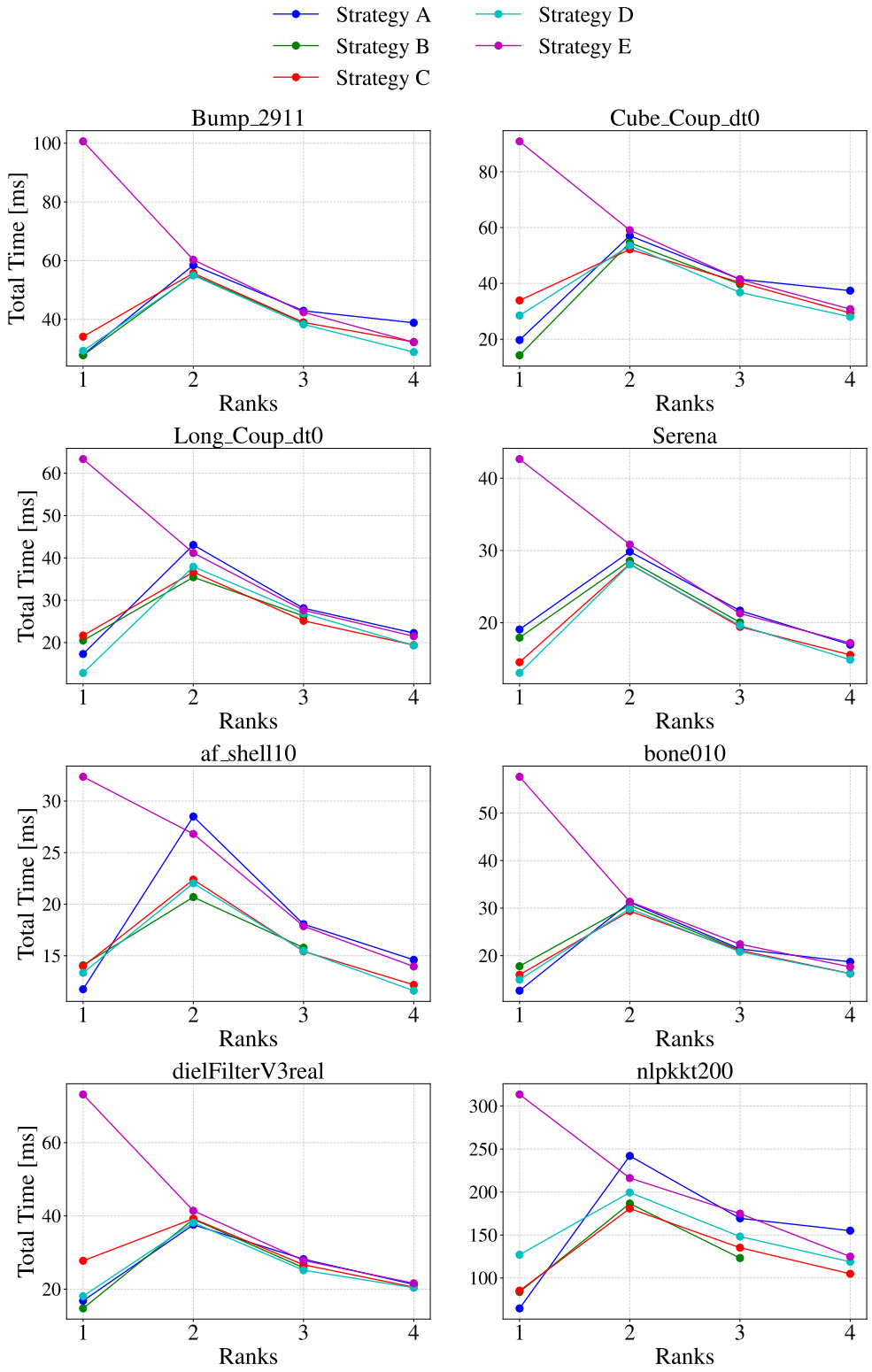


Figure 5.8: Total execution time per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7601 processors.

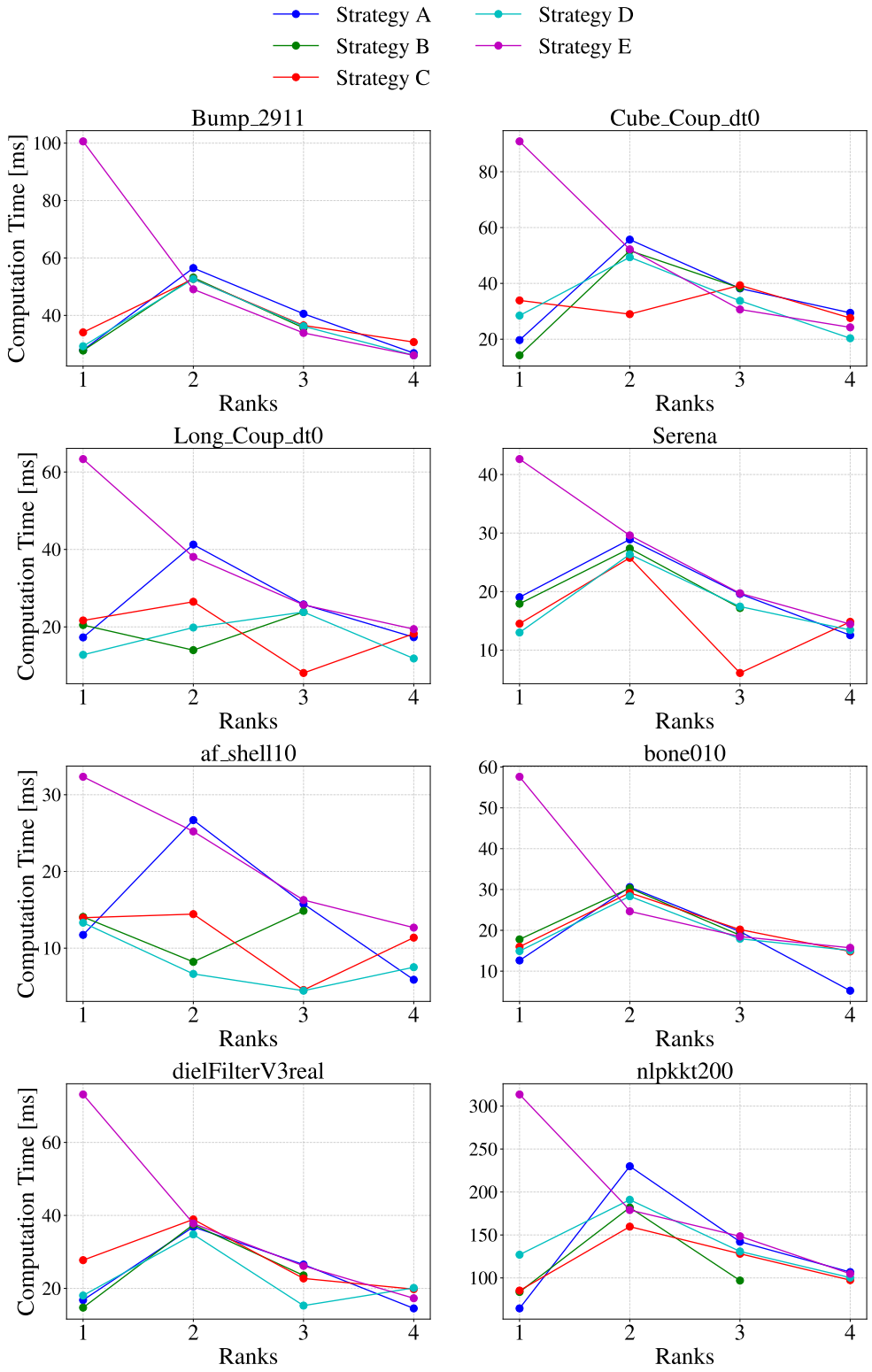


Figure 5.9: Computation time per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7601 processors.

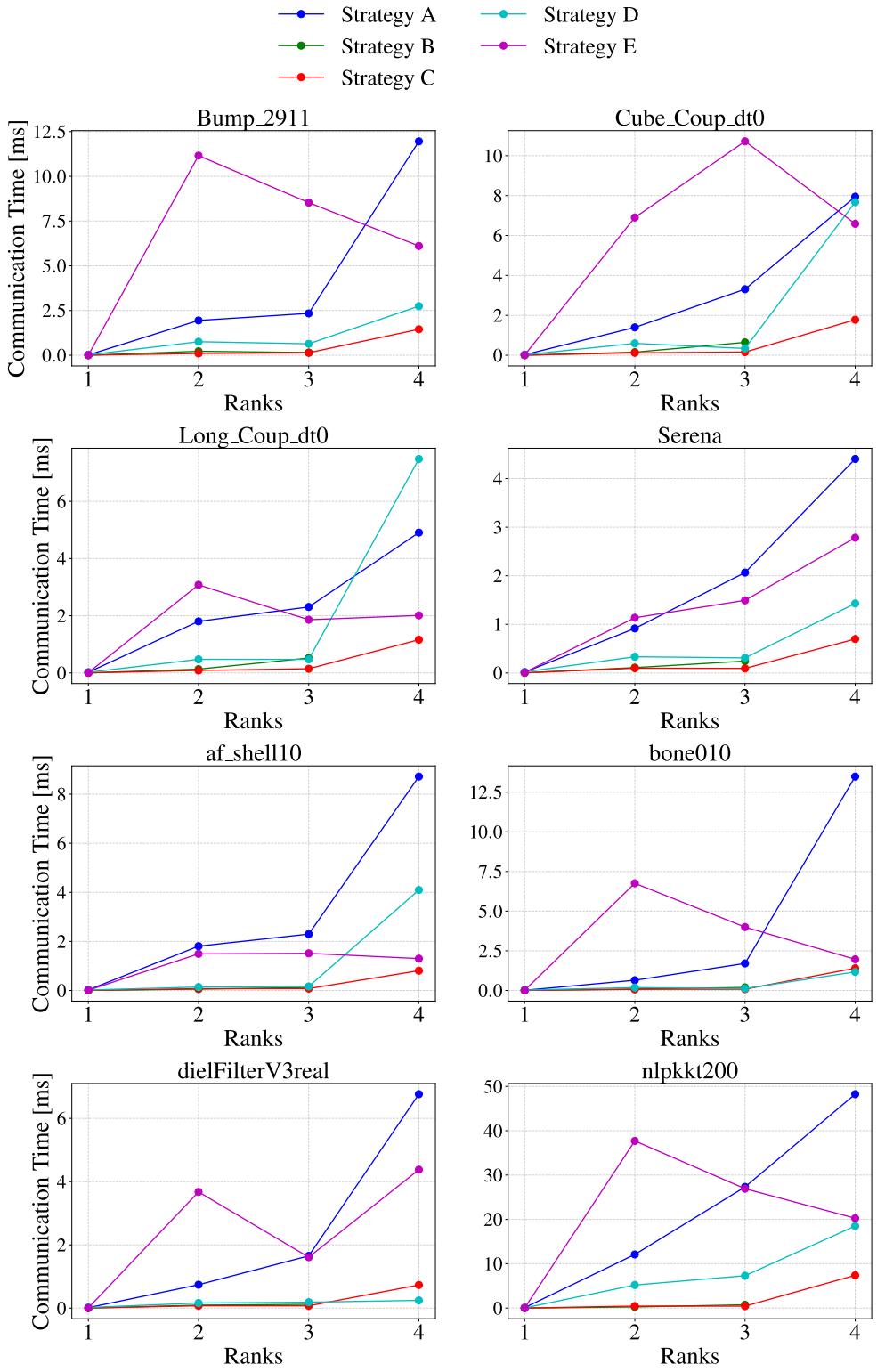


Figure 5.10: Communication time per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7601 processors.

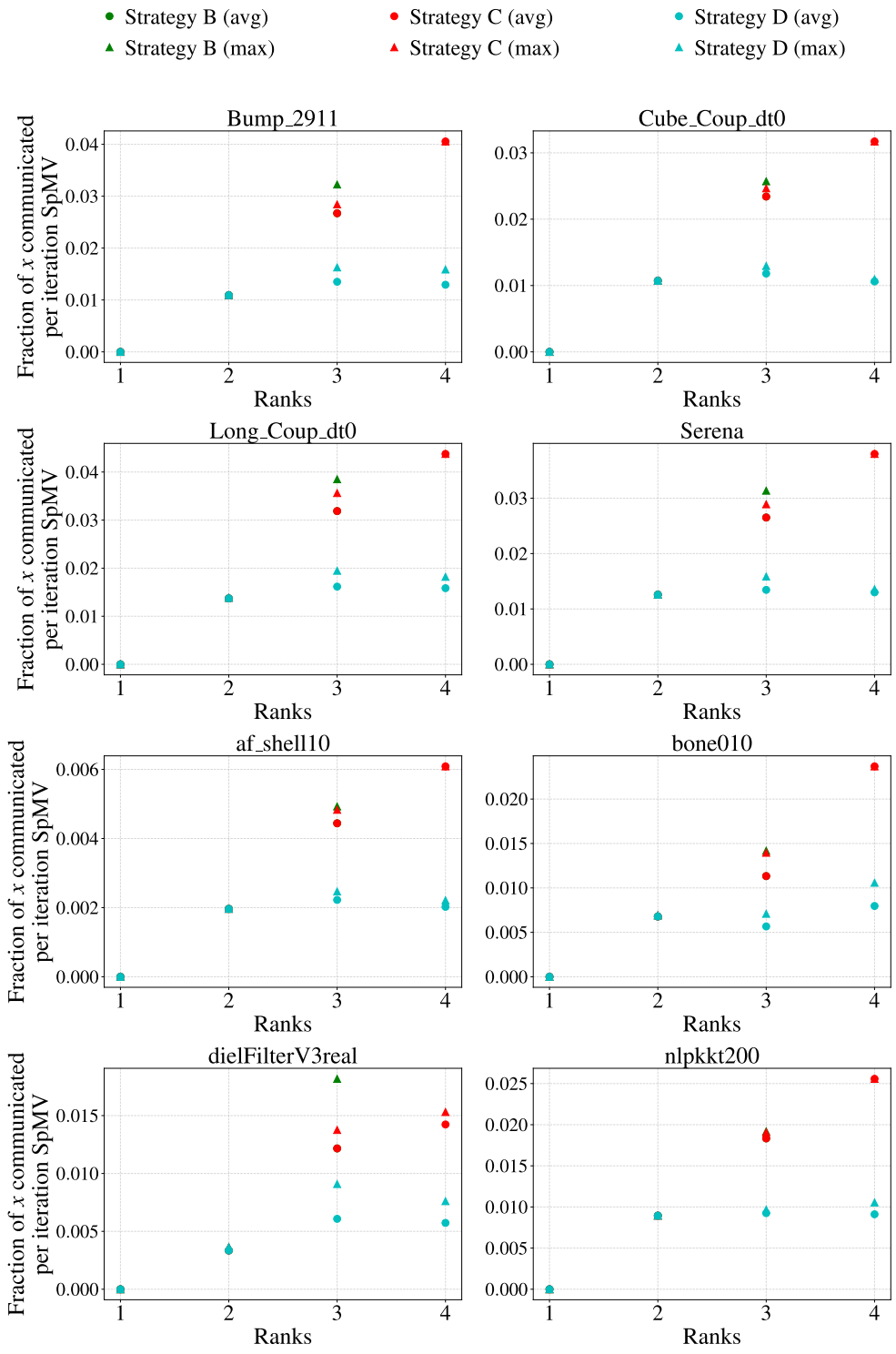


Figure 5.11: Fraction of the global x vector communicated per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7601 processors.

One MPI rank per socket

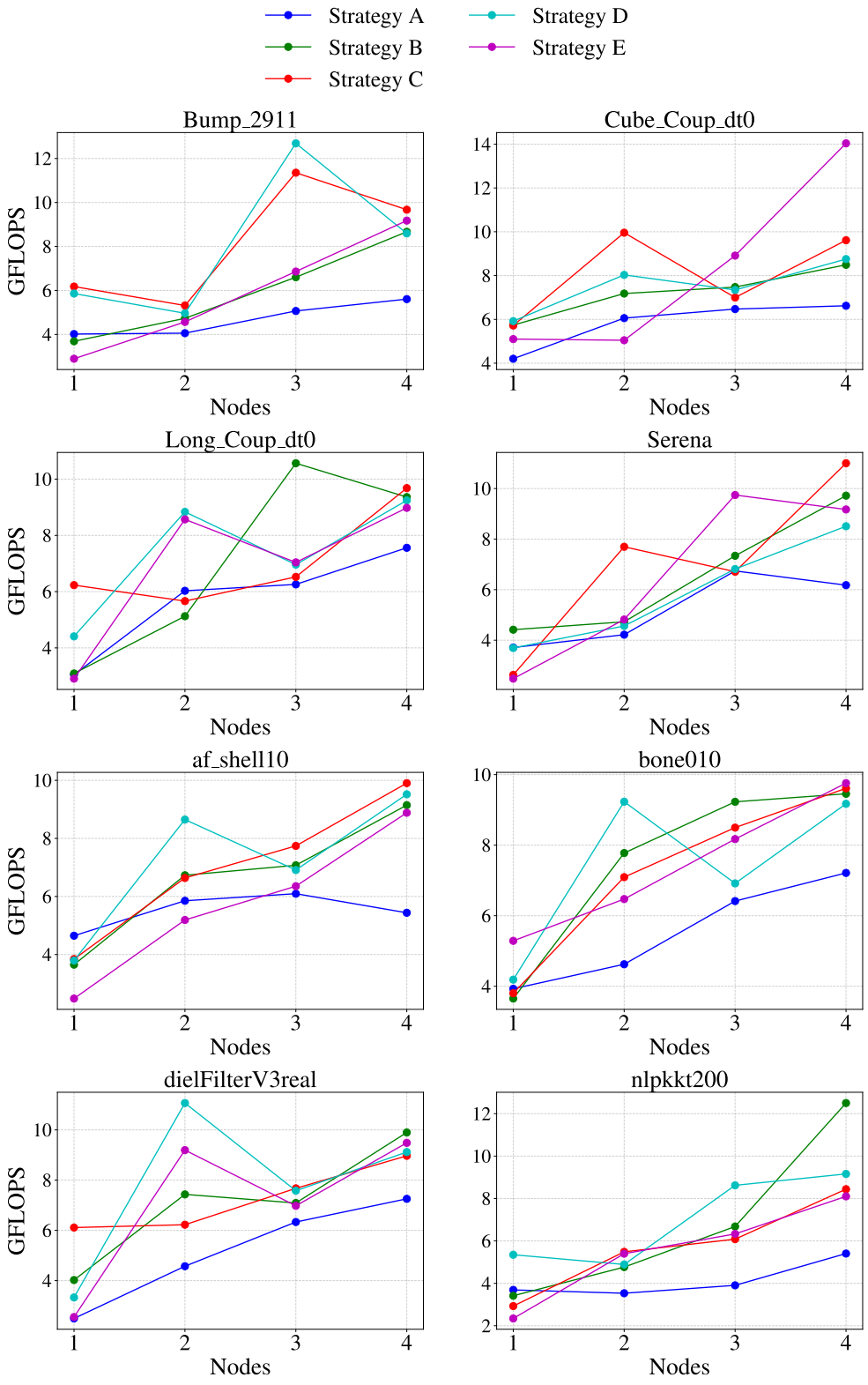


Figure 5.12: Sustained GFLOPS performance of SpMV over 100 iterations on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7601 processors.

Figure 5.12 illustrate the achieved GFLOPS when placing one MPI rank per socket. The benefits of doing this is that we get to utilize the memory controllers on both chips, instead of only one when placing one MPI rank per node.

When compared to Figure 5.7, we see that the performance drops when using only one node has drastically dropped, but that most communication strategies performs better with this configuration when scaling to multiple nodes. However, even when using four nodes, the performance is not as good as using a single node with one MPI rank.

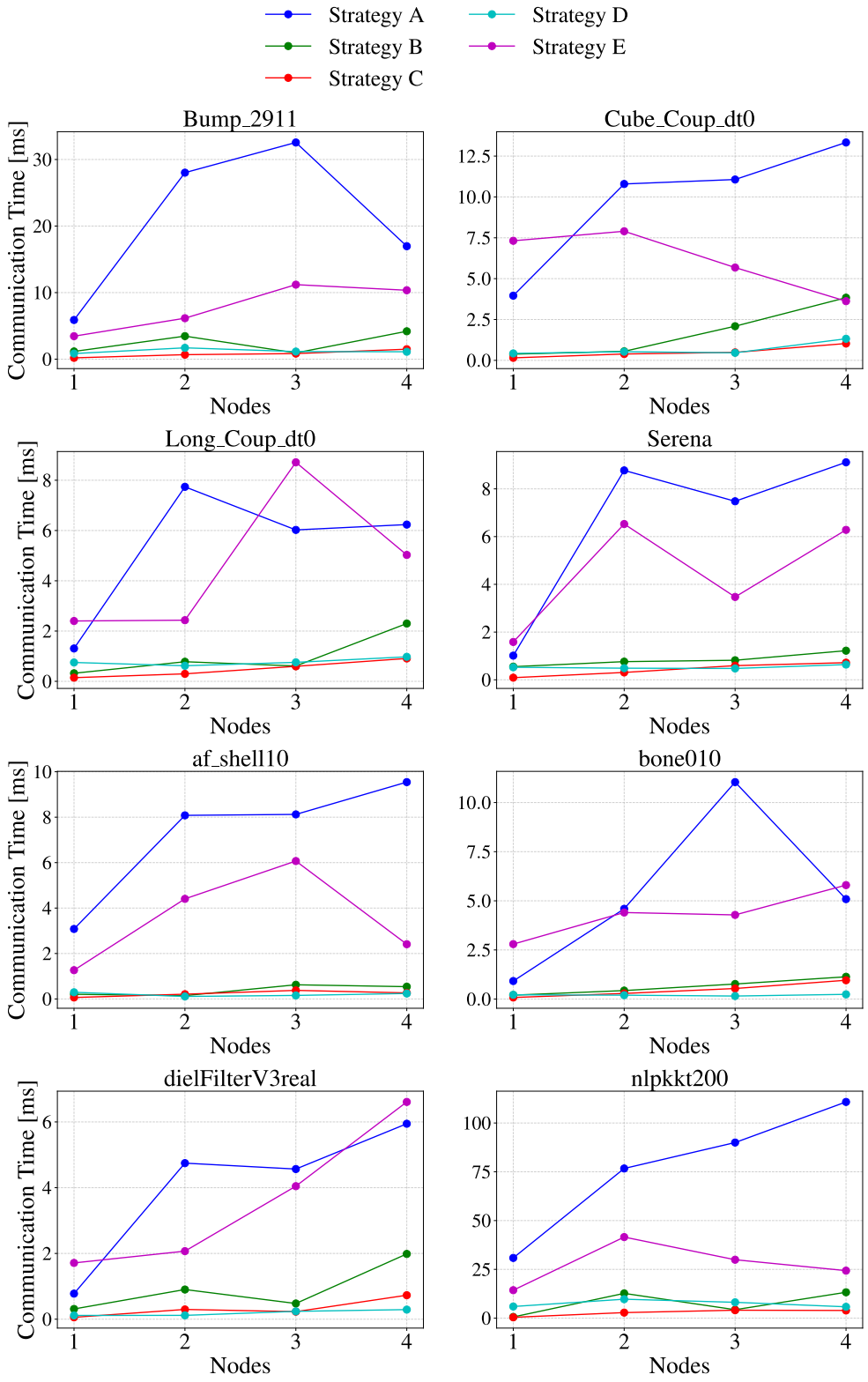


Figure 5.13: Communication time per iteration of SpMV on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7601 processors.

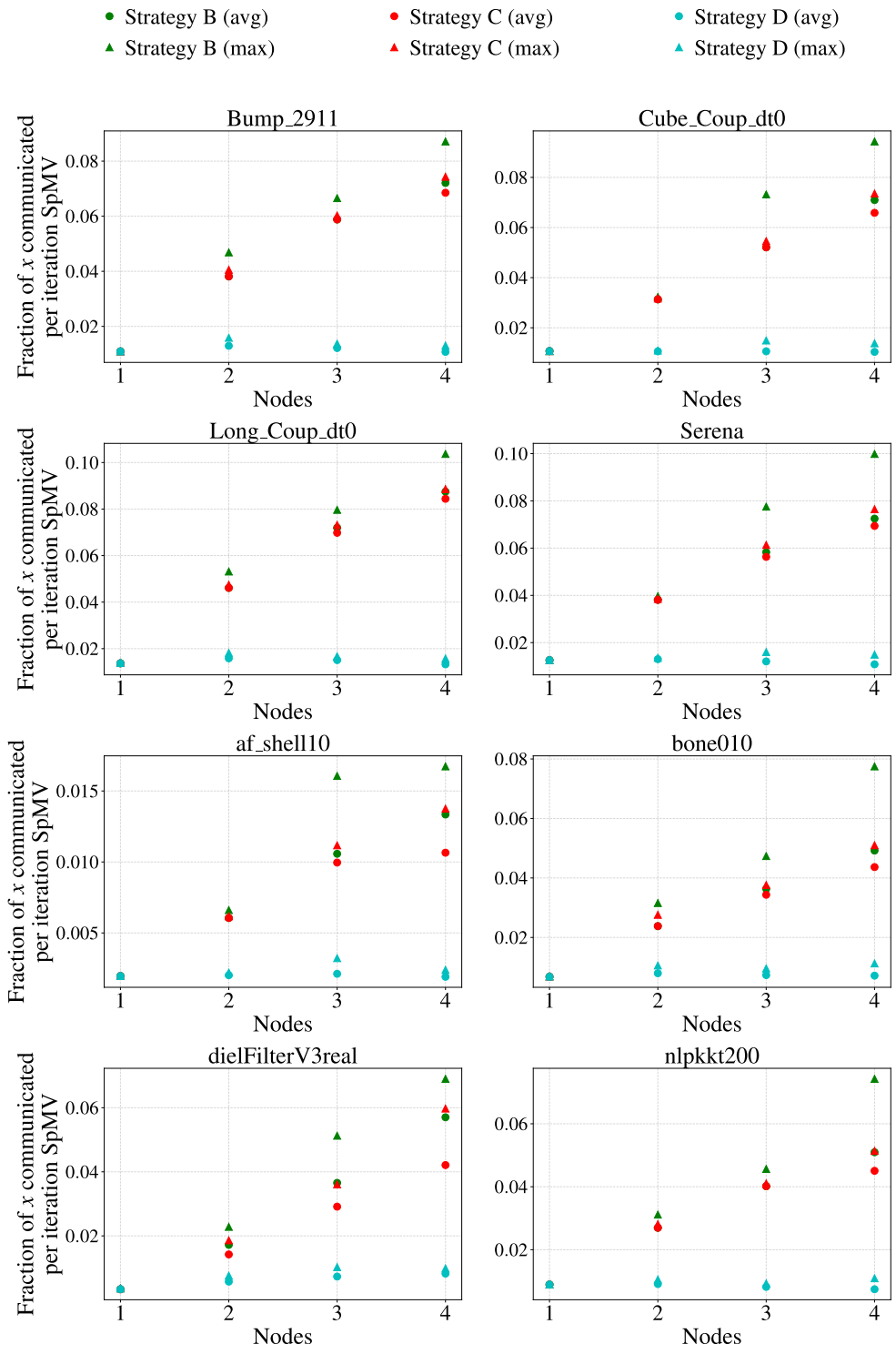


Figure 5.14: Fraction of the global x vector communicated per iteration of SpMV on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7601 processors.

5.4 AMD EPYC 7302P

The results in this section present the performance metrics of experiments ran on 1-8 nodes with a single-socketed AMD EPYC 7302P chip. As the nodes are single socketed, only one set of experiments are provided, where one MPI rank is placed per node, each with 16 OpenMP threads, as the AMD EPYC 7302P chip only has 16 physical cores. Figure 5.15 presents the achieved GFLOPS across up to eight single-socket nodes using the AMD EPYC 7302P processor. These results offer a more fine-grained perspective on the performance of various communication strategies, particularly when employing a non-power-of-two number of MPI ranks. In contrast to the single node experiments discussed in Section 5.3.1, where the number of ranks were limited to powers of two, this linear range of nodes shows how the communication strategies behave under less ideal conditions.

As is evident from the results, strategy E consistenly underperforms compared to all other strategies. This is not necessarily a cause for worry, as the experiments were only ran on up to 8 nodes. This might not be a large enough number of nodes to show the true performance of strategy E as the number of nodes increase beyond 8 nodes.

It is possible that strategy E starts off worse than the other communication strategies, but as the number of nodes increases this strategy will start outperforming the other strategies. This is backed up by the consistent increase in communication volume of strategy B and C shown Figure 5.19.

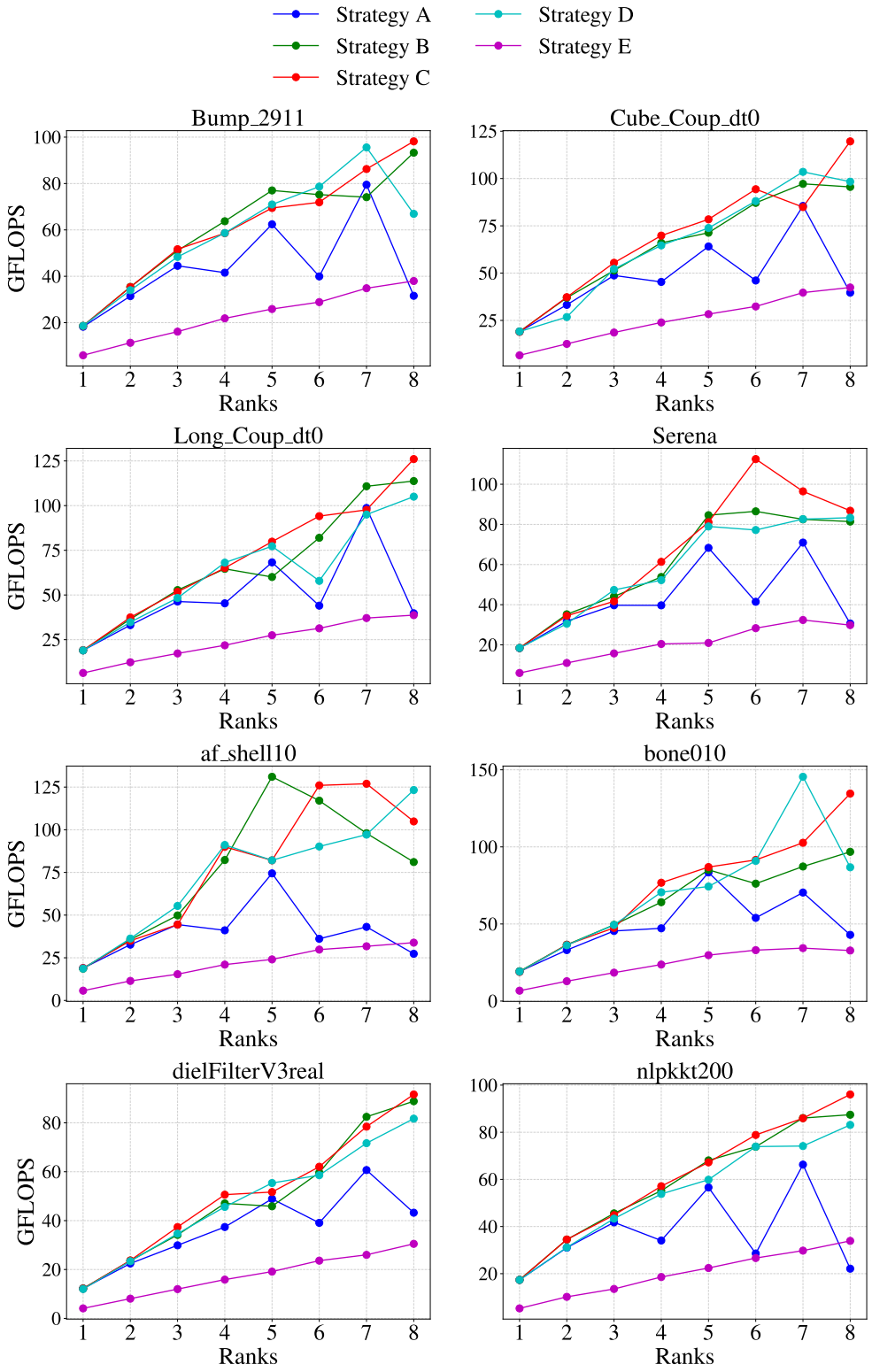


Figure 5.15: Sustained GFLOPS performance of SpMV over 100 iterations on 1–8 nodes (one MPI rank per node) using single-socket AMD EPYC 7302P processors.

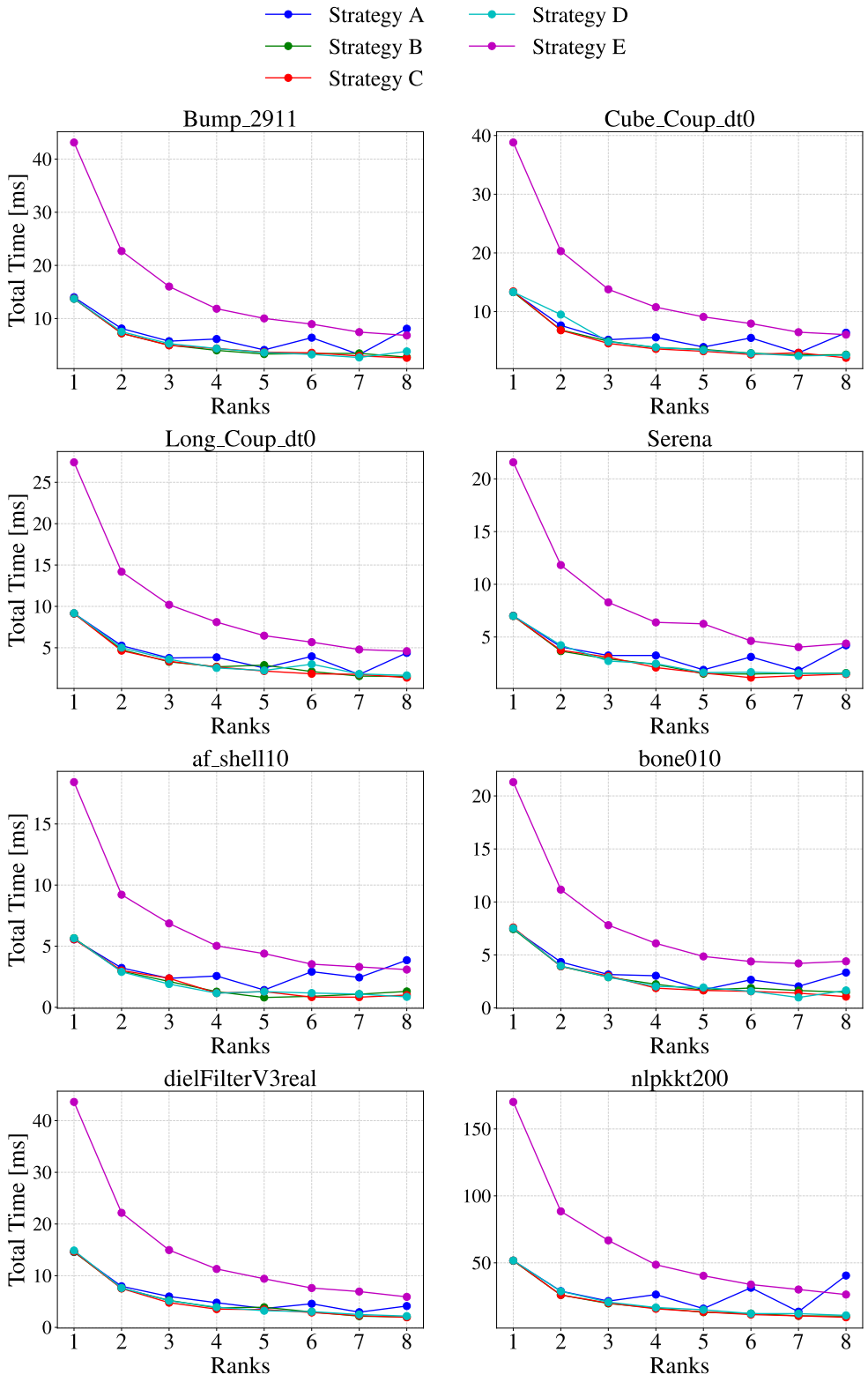


Figure 5.16: Total execution time for 100 iterations of SpMV on 1–8 nodes (one MPI rank per node) using single-socket AMD EPYC 7302P processors.

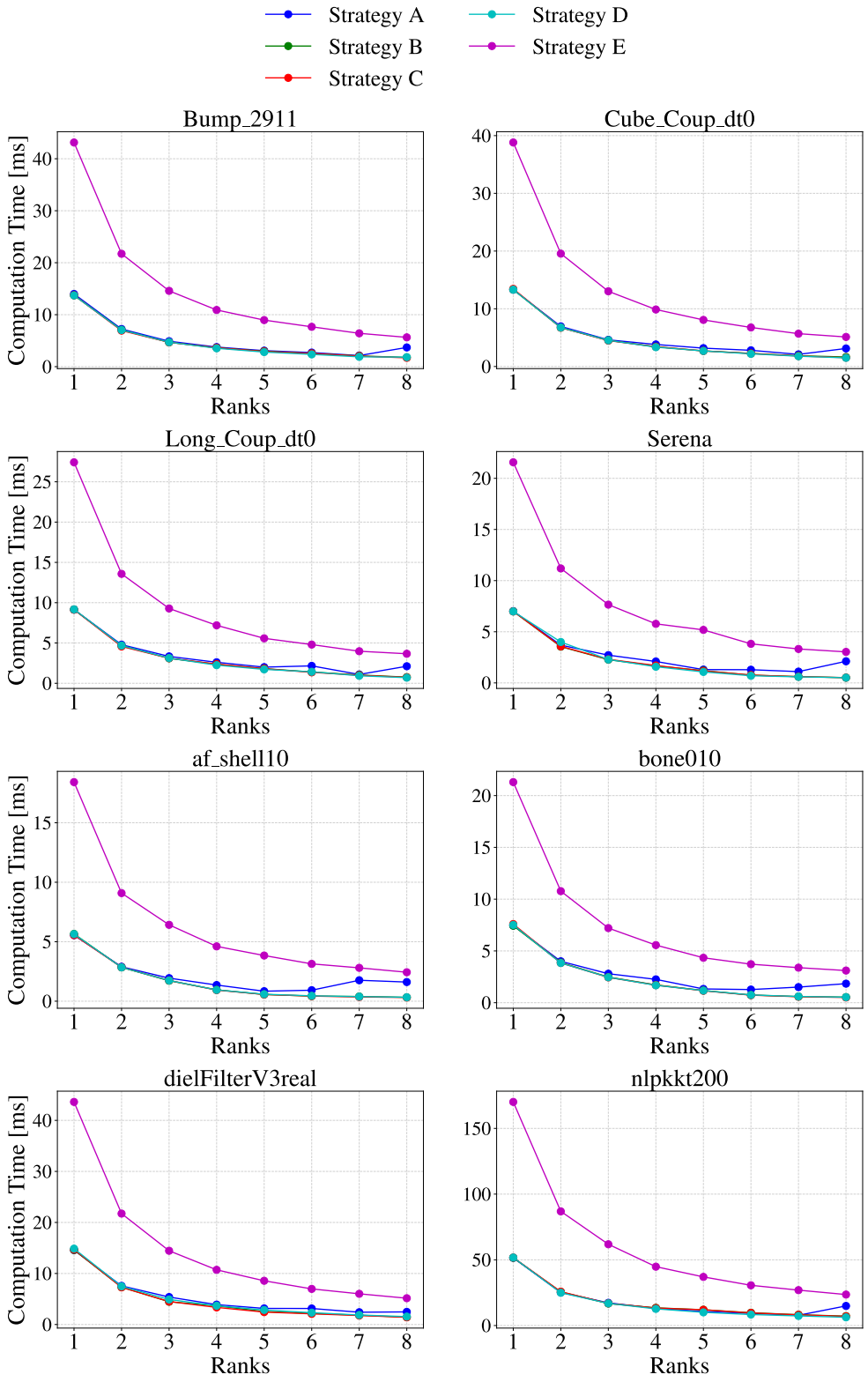


Figure 5.17: Computation time per iteration of SpMV on 1–8 nodes (one MPI rank per node) using single-socket AMD EPYC 7302P processors.

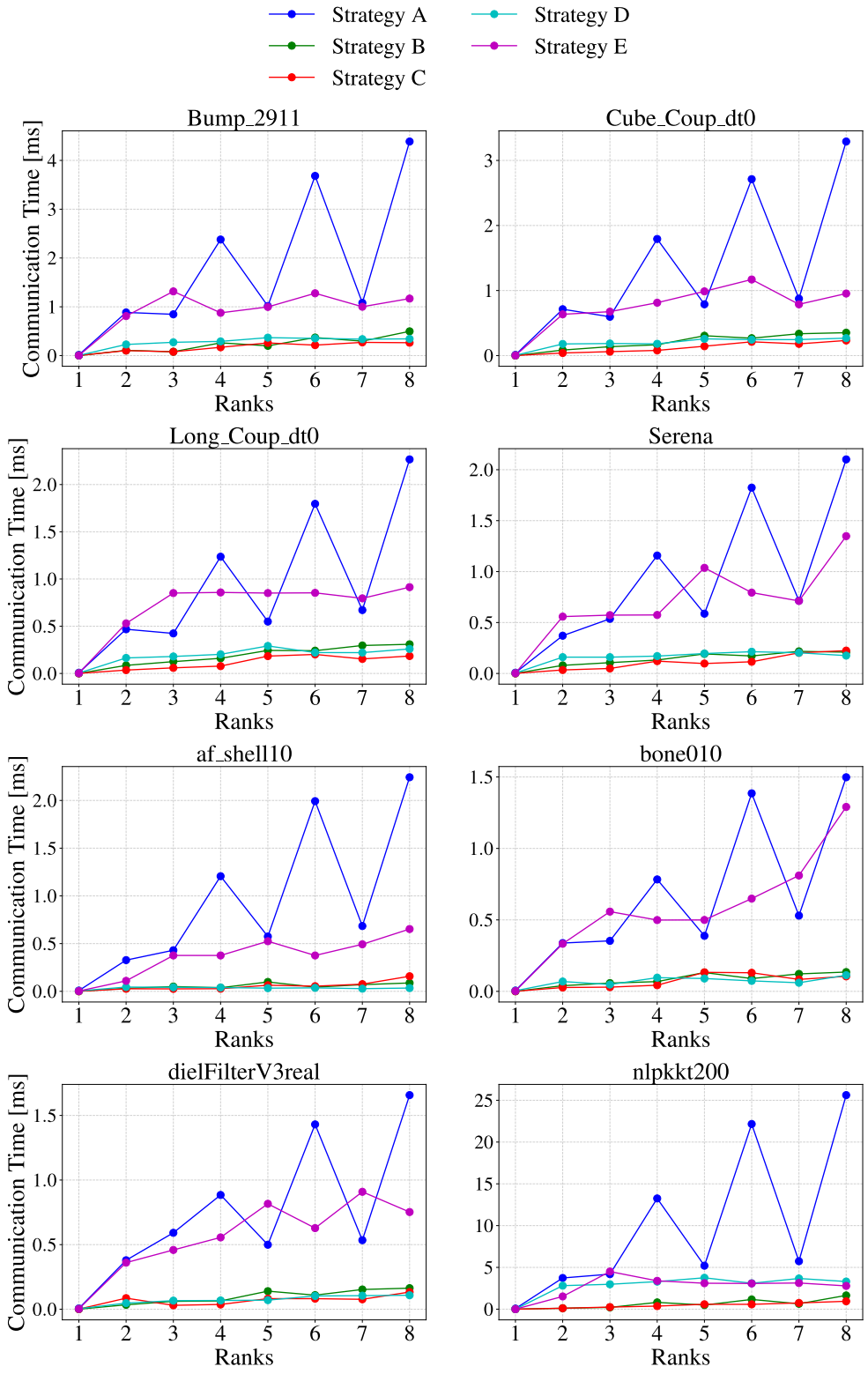


Figure 5.18: Communication time per iteration of SpMV on 1–8 nodes (one MPI rank per node) using single-socket AMD EPYC 7302P processors.

The results in Figure 5.18 present the communication time per iteration of SpMV on up to eight AMD EPYC 7302P nodes. Notably, Strategy A exhibits a sawtooth-like pattern, depending on whether or not the number of nodes used is even or odd. Such irregularities is can be explained by the underlying communication algorithms MPI decides to employ when using `MPI_Allgatherv`. MPI decides on some internal algorithm to use in collective communication routines, depending on the number of ranks involved in the operation and the size of the messages amongst other factors. The exact algorithm MPI uses internally for these results is unknown, and outside the scope of this thesis, but it is the most probable reason that this sawtooth pattern is observed, this is also backed up by the fact that the results in Figure 5.17, which illustrate the computation time for each strategy, don't exhibit this behaviour. Thune et. al conducted a study investigating the modeling of heterogeneous and contention-constrained point-to-point MPI Communication, see [13].

Looking at the single node results in Figure 5.4, strategy D and E performs comparably, however this is not the case here, as strategy E is beaten by strategies B-D in all matrices except for `nlpkkt200`. This discrepancy can be attributed to the cost reindexing the local x vector has on memory locality. In the single node setting, the shared physical memory and fast intra node communication mitigate the effects of these memory access irregularities. However, in the multi node case communication across nodes takes longer

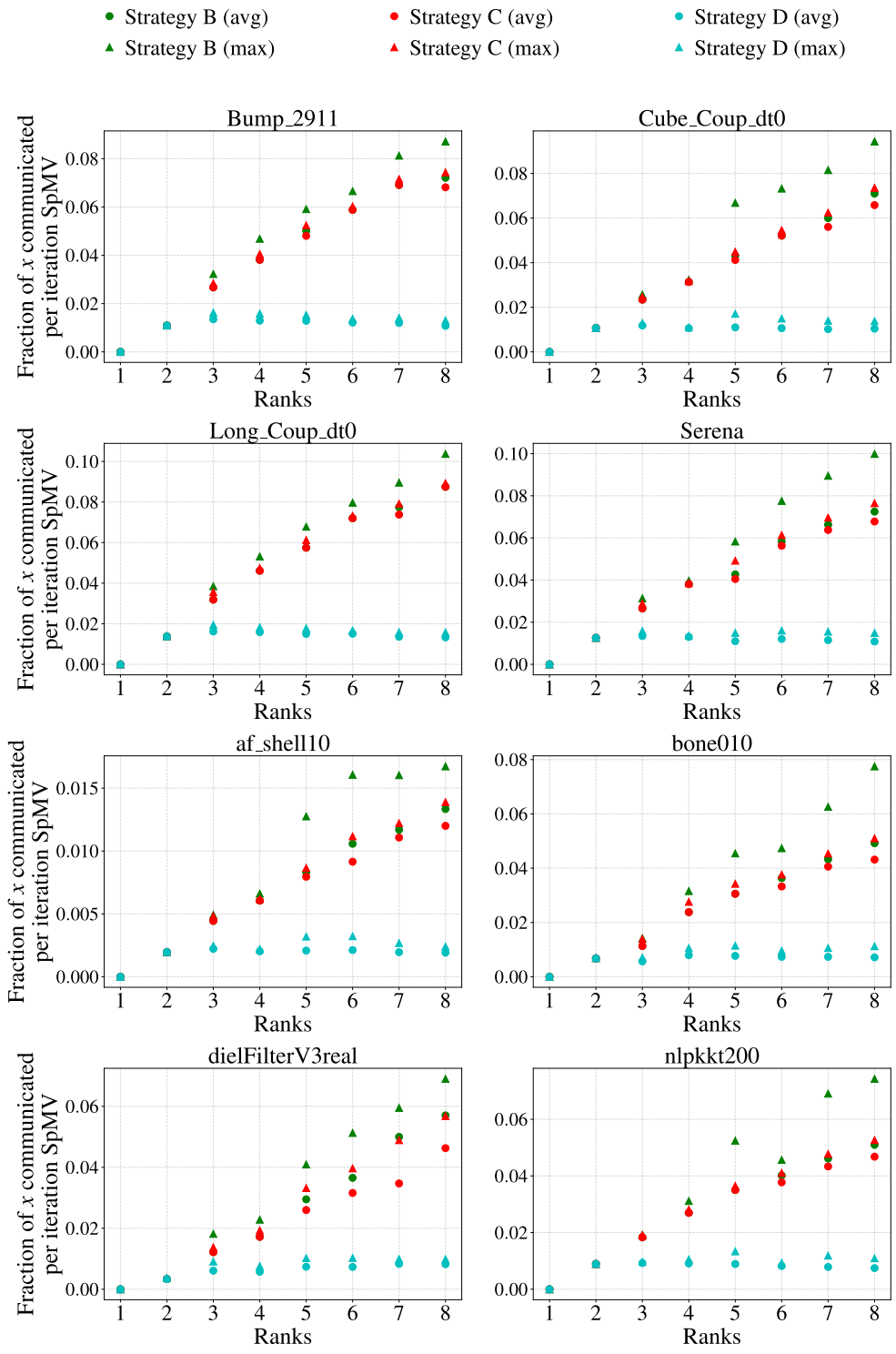


Figure 5.19: Fraction of the global x vector communicated per iteration of SpMV on 1–8 nodes (one MPI rank per node) using single-socket AMD EPYC 7302P processors.

5.5 AMD EPYC 7413

Two sets of experiments were conducted on the AMD EPYC 7413 nodes. As these nodes are also dual-socketed, the experiments conducted are similar to those on the AMD EPYC 7601 nodes, but they only utilize 24/48 OpenMP threads per socket/node, as the AMD EPYC 7413 chip provides 24 physical cores.

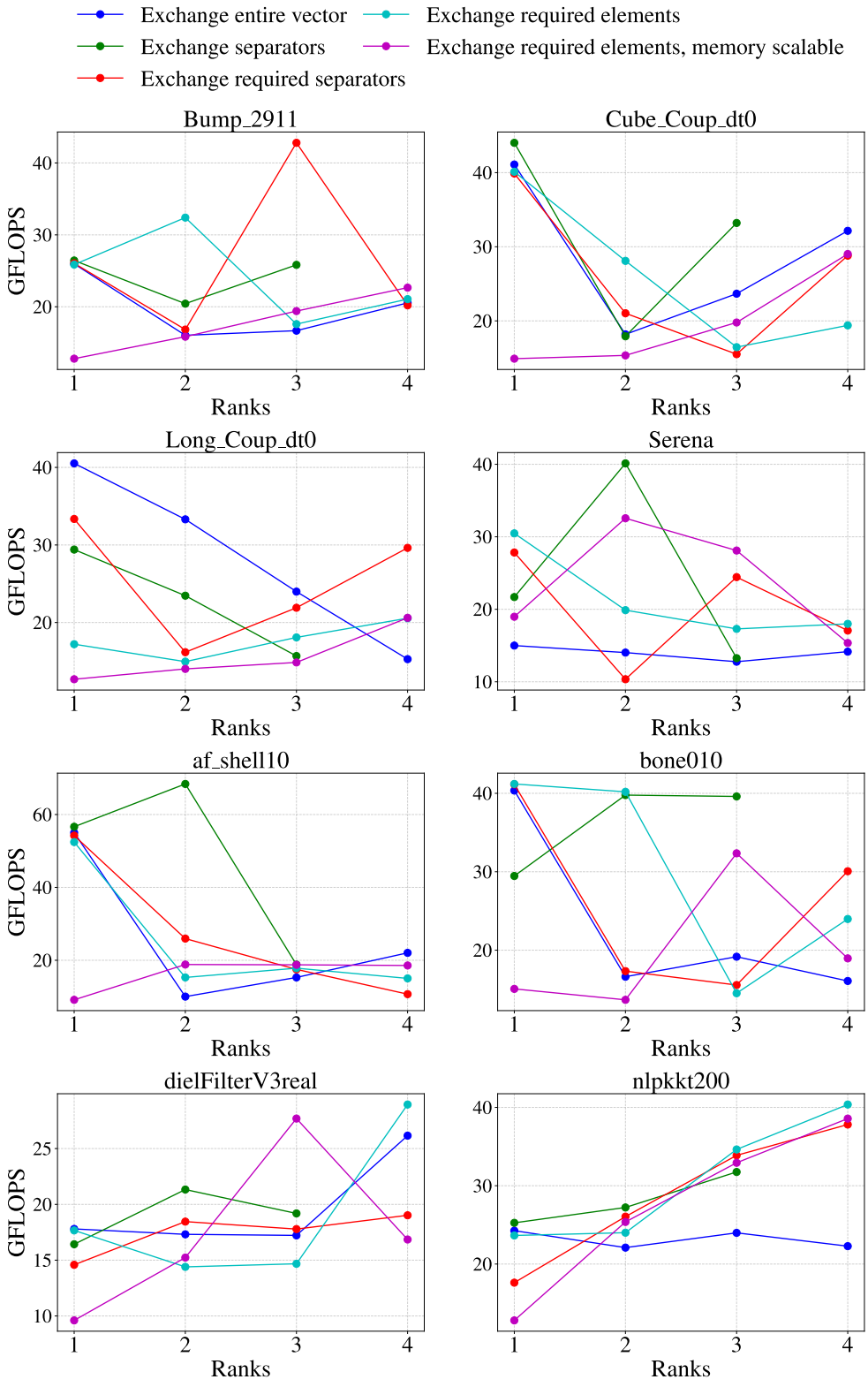


Figure 5.20: multi Node - AMD EPYC 7413

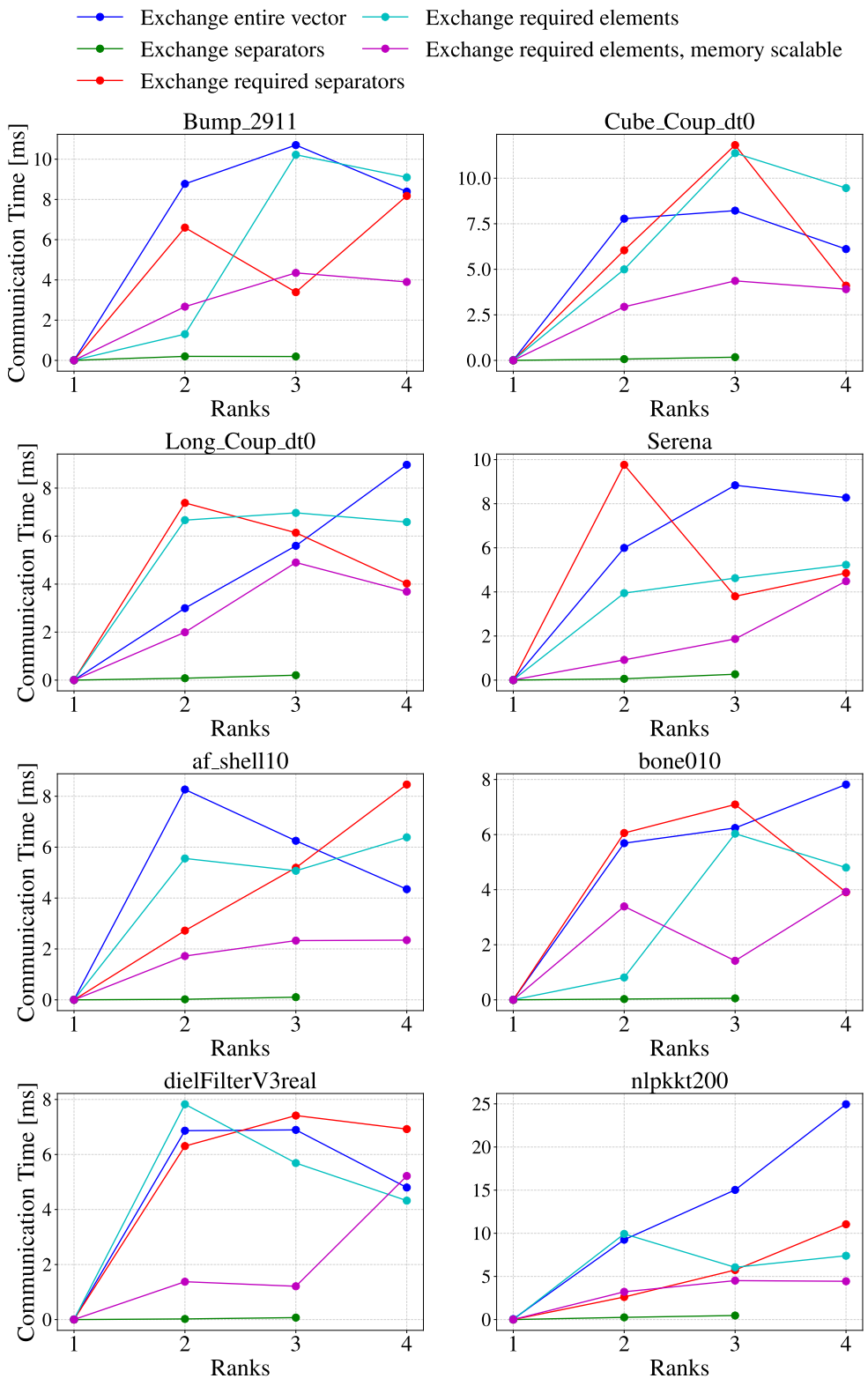

Exchange entire vector
Exchange separators
Exchange required separators
Exchange required elements
Exchange required elements, memory scalable
Bump_2911
Cube_Coup_dt0
Long_Coup_dt0
Serena
af_shell10
bone010
dielFilterV3real
nlpkkt200
Communication Time [ms]
Ranks

Figure 5.21

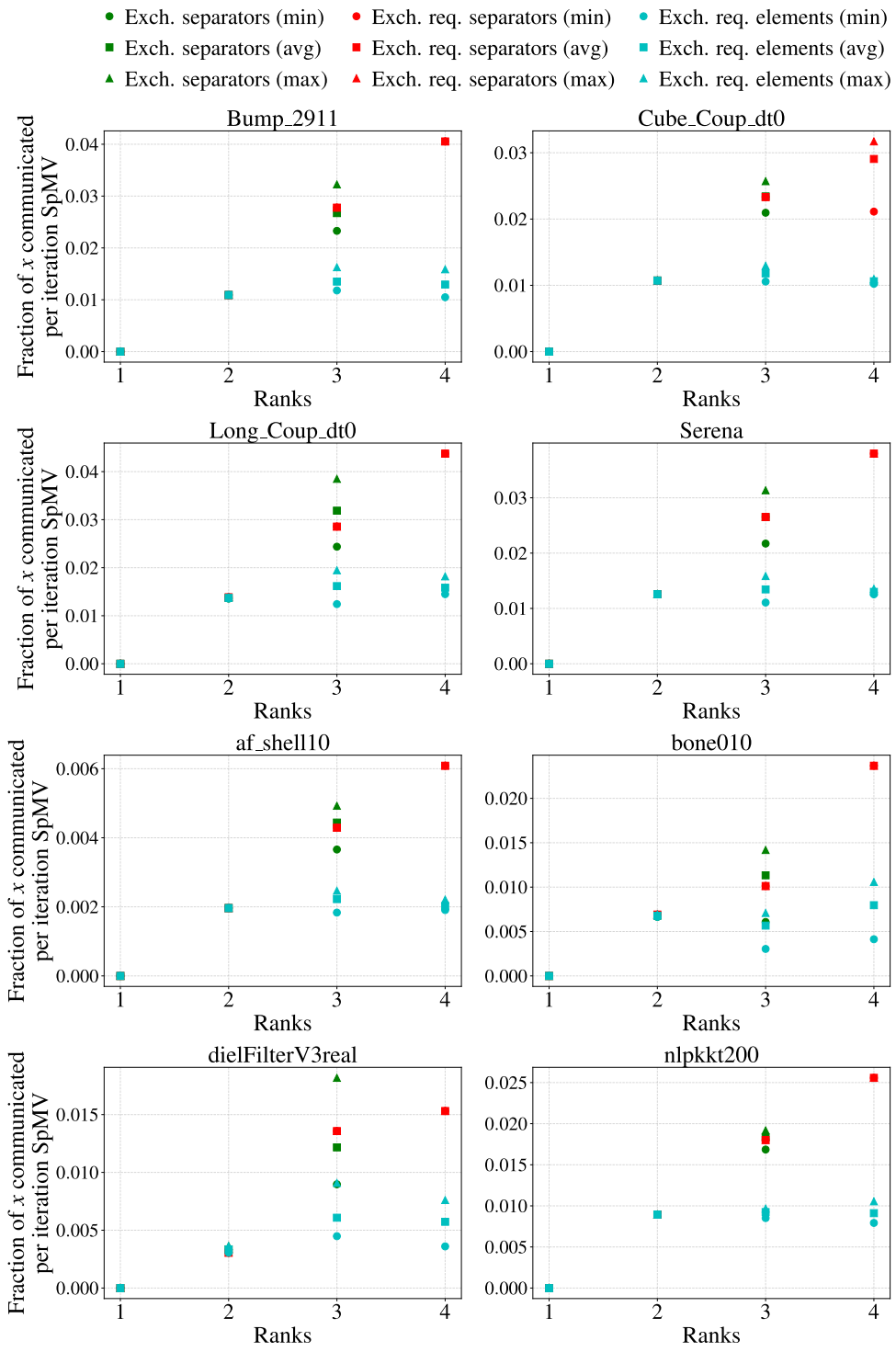

Exch. separators (min)
Exch. separators (avg)
Exch. separators (max)
Exch. req. separators (min)
Exch. req. separators (avg)
Exch. req. separators (max)
Exch. req. elements (min)
Exch. req. elements (avg)
Exch. req. elements (max)
Bump_2911
Cube_Coup_dt0
Long_Coup_dt0
Serena
af_shell10
bone010
dielFilterV3real
nlpkkt200
Fraction of x communicated per iteration SpMV
Ranks

Figure 5.22

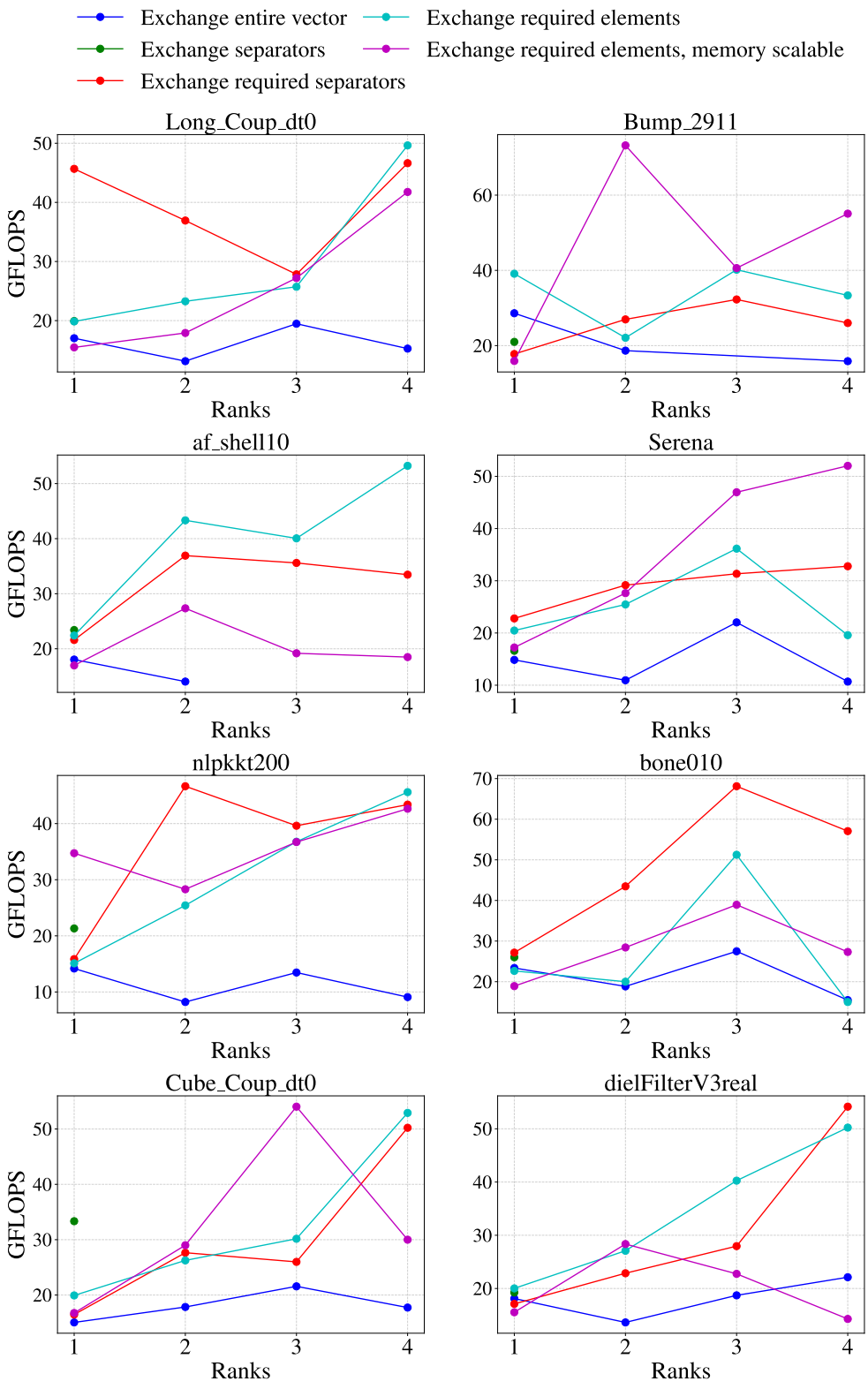


Figure 5.23: Multi Node - AMD EPYC 7413

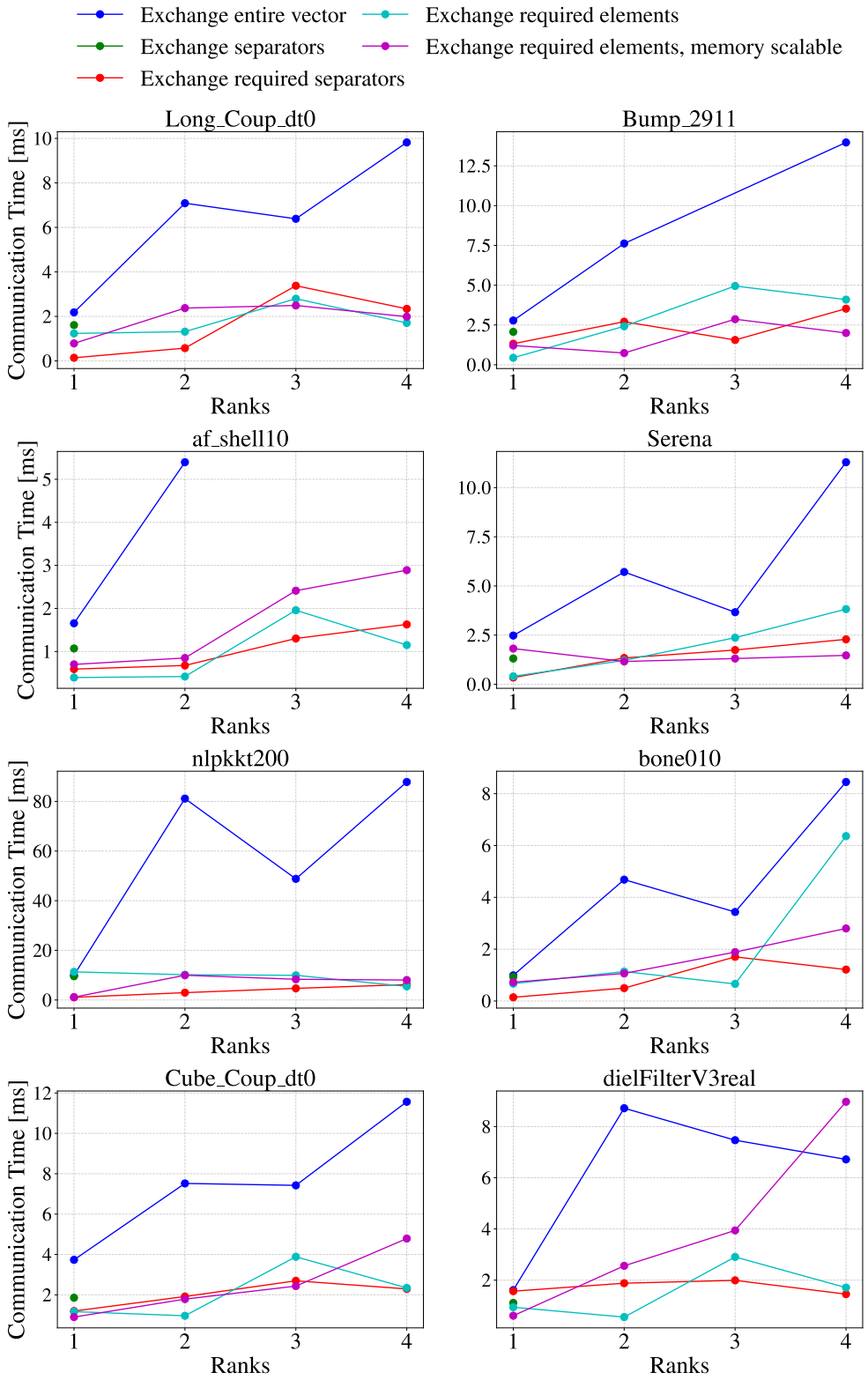

Exchange entire vector
Exchange separators
Exchange required separators
Exchange required elements
Exchange required elements, memory scalable
Long_Coup_dt0
Bump_2911
af_shell10
Serena
nlpkkt200
bone010
Cube_Coup_dt0
dielFilterV3real
Communication Time [ms]
Ranks

Figure 5.24

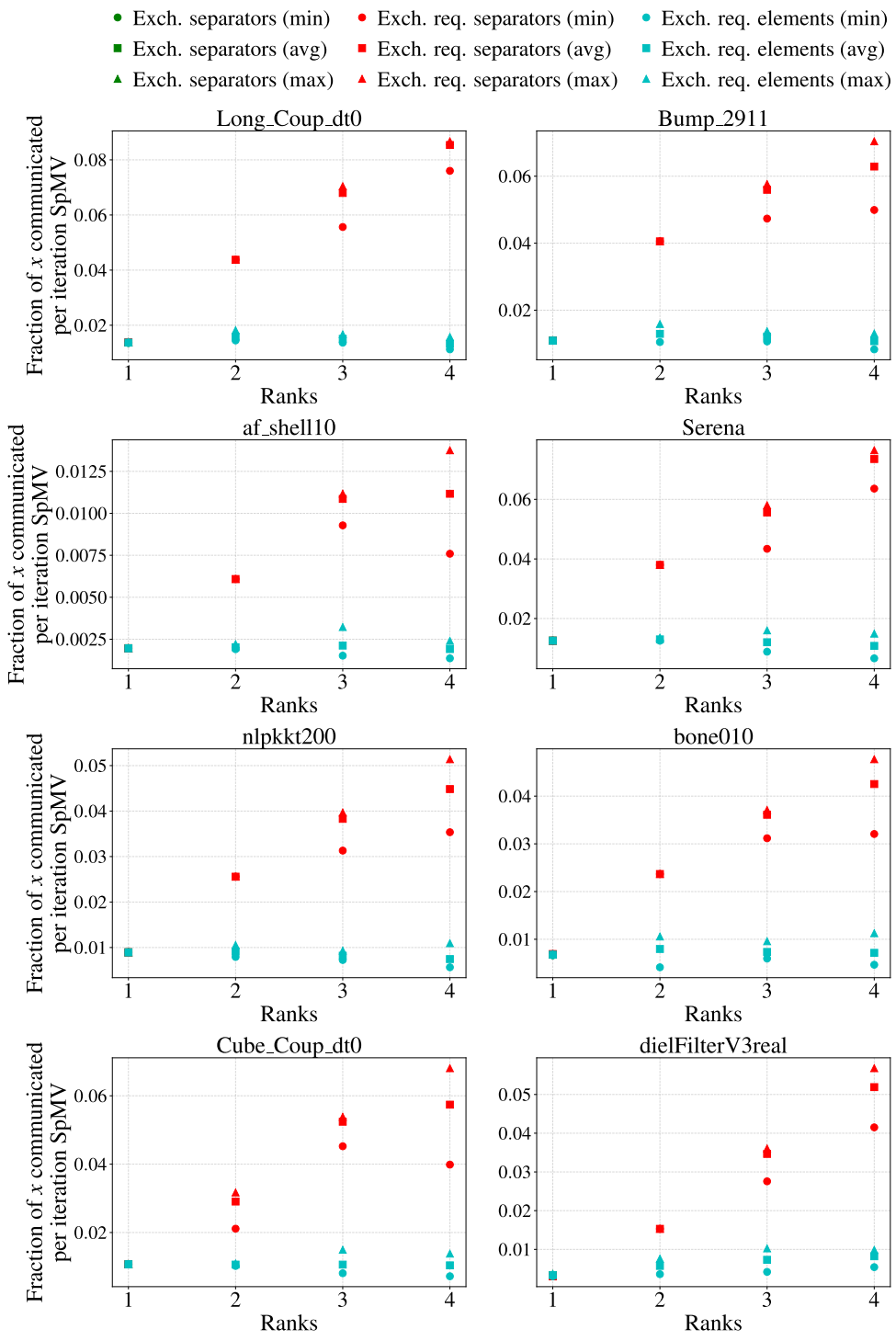

Exch. separators (min)
Exch. separators (avg)
Exch. separators (max)
Exch. req. separators (min)
Exch. req. separators (avg)
Exch. req. separators (max)
Exch. req. elements (min)
Exch. req. elements (avg)
Exch. req. elements (max)
Long_Coup_dt0
Bump_2911
af_shell10
Serena
nlpkkt200
bone010
Cube_Coup_dt0
dielFilterV3real
Fraction of x communicated per iteration SpMV
Ranks

Figure 5.25

Chapter 6

Conclusion

CONCLUSION GOES HERE

Chapter 7

Related Work

SpMV has been a central topic of research in high performance computing due to its widespread use in iterative solvers and numerical simulations. Over the past decades, various approaches have been proposed to optimize SpMV on both shared and distributed memory systems, focusing on reducing communication overhead, improving memory access locality, and achieving effective load balancing. The inherent sparsity and irregularity of memory access patterns in SpMV make it particularly challenging to parallelize efficiently, especially at scale. Consequently, significant efforts have been devoted to designing storage formats, partitioning strategies, and communication models that better leverage modern hardware architectures. This section provides an overview of key contributions in this area, with an emphasis on partitioning techniques, memory and communication models, and recent innovations in hybrid parallelism.

Merrill and Garland [10] propose a merge-based parallel SpMV algorithm that operates directly on the Compressed Sparse Row (CSR) format without requiring reformatting or auxiliary data structures. Their method employs a 2D merge-path decomposition that ensures strictly balanced workloads across processing elements, independent of row length variability. This design addresses the primary bottleneck of irregular row distributions in parallel SpMV, which often lead to performance inconsistencies in traditional row- and nonzero-splitting strategies. Evaluated across over 4,000 matrices, their approach demonstrated superior performance consistency and scalability, particularly on architectures with constrained local memories such as NUMA and GPUs. This work offers a compelling alternative to format-specific optimizations by achieving portability and high throughput directly on CSR representations.

Trotter et al. [14] present a comprehensive evaluation of matrix reorder-

ing strategies for enhancing sparse matrix-vector multiplication (SpMV) performance on multicore CPUs. Evaluating six reordering algorithms over 490 matrices on eight architectures, they find that reorderings based on graph and hypergraph partitioning (e.g., METIS, PaToH) offer the most consistent SpMV performance benefits. Their results indicate that performance improvements arise primarily from better data locality and reduced off-diagonal nonzero density, while bandwidth and profile reductions are less influential. They also underscore the architectural sensitivity of reordering effectiveness, suggesting that careful algorithm-architecture matching is critical. This study provides empirical validation for the effectiveness of reordering as a pre-processing optimization for SpMV, especially in shared-memory systems.

Burchard et al. [1] explored the implementation of unstructured mesh computations on massively tiled processors, specifically targeting the Graphcore IPU architecture. They ported a cardiac electrophysiology solver, which relies heavily on repeated SpMVs and per cell ODE integrations, to the IPU platform. The paper highlights how Poplar/C++ was utilized to create per-tile data structures that efficiently manage inter tile communication necessary for SpMV parallelization. Their results demonstrate both the feasibility and performance benefits of such architectures for irregular scientific computations, providing insight into parallel execution and data distribution on many core processors without shared memory.

Bibliography

[1] Luk Burchard, Kristian Gregorius Hustad, Johannes Langguth, and Xing Cai. Enabling unstructured-mesh computation on massively tiled ai processors: An example of accelerating in silico cardiac simulation. *Frontiers in Physics*, Volume 11 - 2023, 2023.

[2] U.V. Catalyurek and C. Aykanat. Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication. *IEEE Transactions on Parallel and Distributed Systems*, 10(7):673–693, 1999.

[3] Timothy A. Davis and Yifan Hu. The university of florida sparse matrix collection. *ACM Trans. Math. Softw.*, 38(1), December 2011.

[4] Gautam Gupta, Sivasankaran Rajamanickam, and Erik G. Boman. GAMGI: Communication-reducing algebraic multigrid for gpus. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 61–75. ACM, 2024.

[5] Cerebras Systems Inc. Wse-3 datasheet. https://cerebras.net/product/system/, 2024. DS03 v3 821.

[6] George Karypis and Vipin Kumar. METIS: A Software Package for Partitioning Unstructured Graphs, Partitioning Meshes, and Computing Fill-Reducing Orderings of Sparse Matrices. Technical report, University of Minnesota, Department of Computer Science / Army HPC Research Center, 1997. Version 3.0.3.

[7] Christoph Lameter. Numa (non-uniform memory access): An overview: Numa becomes more common because memory controllers get close to execution units on microprocessors. *Queue*, 11(7):40–51, July 2013.

[8] Lawrence Livermore National Laboratory. Introduction to parallel computing tutorial. http://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial, 2024. Accessed: 2025-05-08.

[9] Nakul Manchanda and Karan Anand. Non-uniform memory access (numa). *New York University*, 4, 2010.

[10] Duane Merrill and Michael Garland. Merge-based parallel sparse matrix-vector multiplication. In *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 678–689, 2016.

[11] Peter Norvig. Latency numbers every programmer should know. https://norvig.com/21-days.html#Latency, 2021. Accessed: 2025-04-29.

[12] P. Stenstrom. A survey of cache coherence schemes for multiprocessors. *Computer*, 23(6):12–24, 1990.

[13] Andreas Thune, Sven-Arne Reinemo, Tor Skeie, and Xing Cai. Detailed modeling of heterogeneous and contention-constrained point-to-point mpi communication. *IEEE Transactions on Parallel and Distributed Systems*, 34(5):1580–1593, 2023.

[14] James D. Trotter, Sinan Ekmekçibaşı, Johannes Langguth, Tugba Torun, Emre Düzakın, Aleksandar Ilic, and Didem Unat. Bringing order to sparsity: A sparse matrix reordering study on multicore cpus. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '23, New York, NY, USA, 2023. Association for Computing Machinery.

[15] Cong Zheng, Shuo Gu, Tong-Xiang Gu, Bing Yang, and Xing-Ping Liu. Biell: A bisection ellpack-based storage format for optimizing spmv on gpus. *Journal of Parallel and Distributed Computing*, 74(7):2639–2647, 2014. Special Issue on Perspectives on Parallel and Distributed Processing.